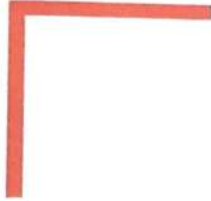


7장 3차원 뷰잉



학습 목표

1. 3차원 뷰잉 파이프라인을 설명할 수 있다.
2. 3차원 뷰잉 좌표를 설정하기 위한 여러 가지 파라미터를 지정할 수 있다.
3. 평행투영과 원근투영의 다양한 형태를 설명할 수 있다.
4. 평행투영과 원근투영을 위한 파라미터를 설정할 수 있다.
5. OpenGL을 이용하여 3차원 장면을 표현할 수 있다.

주요 용어

평행투영 / 원근투영 / 깊이단서기법 / 뷰 평면 / 뷰 체적 / 뷰 지점 / 뷰 원점 / 뷰업 벡터 /
뷰 평면 법선 벡터 / 직교투영 / 축각직교투영 / 등각투영 / 경사투영 / 캐버리어 투영 /
캐비닛 투영 / 소실점 / 1점 투영 / 2점 투영 / 3점 투영



7.1 3차원 뷰잉 개요

2차원 평면에서 고려하지 않았지만, 3차원 공간에서는 고려할 사항이 여러 가지 있다. 우선 3차원 뷰잉 과정에서는 관찰자의 위치와 관찰 방향을 지정해야 한다. 또한 묘사된 3차원 장면이 카메라의 필름과 같은 기능의 뷰 평면에 투영되어야 하며, 조명과 물체의 관계에서 표면 렌더링(surface rendering) 기법을 적용해야만 현실감 있는 3차원 컴퓨터 그래픽 화면을 얻을 수 있다.

7.1.1. 3차원 뷰잉 고려사항

3차원 공간의 입체가 2차원 평면인 뷰 평면(view plane)에 표시되기 위해서는 투영과정이 필요하다. 투영은 용도에 따라 크게 공학 또는 건축 설계에서 물체를 나타낼 때 사용되는 기법인 평행투영(parallel projection)과 소실점을 따라 뷰 평면에 물체를 투영하여 먼 곳의 물체가 가까운 곳의 물체보다 작게 표시되는 원근투영(perspective projection)으로 구별되는데, 용도에 따라 적절한 기법을 선택할 수 있다.

3차원 장면을 화면에 표시하기 위해서는 관찰 방향에 따라 깊이 정보를 도출하여 앞에 놓인 물체와 뒤에 놓인 물체를 판단하는 과정이 필요하다. 깊이 정보를 표현하는 방법에는 여러 가지가 있는데, 선형구조에서 깊이를 나타내는 간단한 방법으로는 깊이 단서기법(depth cueing)이 있다. 이 방법은 관찰지점과의 거리에 따라 선분의 밝기를 변화시키는 것으로 관찰지점에 가장 가까운 선분을 가장 밝게 표시하고, 점점 멀어질수록 약하게 표시하는 것이다. 깊이 정보를 표현하는 또 다른 방법은 대기현상에 의해 인간이 인식하는 밝기 현상을 그대로 표현하는 것으로, 가까운 곳의 물체보다 먼 거리의 물체를 희미하게 표현하는 것이다. 어떤 경우는 대기현상으로 물체의 색상이 바뀌어 인식되기도 하므로 이러한 것을 이용하여 깊이 정보를 표현할 수 있다.

보이는 부분과 가려진 부분을 구별하여 표시하는 과정도 필요하다. 이것은 깊이 정보를 활용하여 판단할 수 있다. 눈에 보이는 선을 밝은 선이나 다른 색으로 표시하거나, 보이지 않는 선을 점선으로 표시하는 방법을 사용할 수 있다. 또는 보이지 않는 선을 제거하는 방법도 있다.

해당 장면의 조명 조건과 표면의 재질 특성을 물체 표면에 반영하는 표면 렌더링 과정도 필요하다. 이러한 표면 연출을 통해 좀 더 현실감 있는 장면을 연출할 수 있다. 조명 조건은 위치와 색상을 고려해야 하고, 경우에 따라서는 부분조명과 배경조명 효과도 고려해야 한다. 물체의 표면 특성은 표면이 투명 또는 불투명인가, 매끄러운가 또는 거친가 등의 정보를 포함한다. 현실감 있는 장면을 표현하기 위해 표면 렌더링 기법과 투영, 그리고 보이는 면까지 확인하여 결합시키는 작업이 필요하다.

7.1.2. 3차원 뷰잉 파이프라인

3차원 장면을 만들고 그것을 컴퓨터 모니터에 생동감 있게 나타내는 과정은 카메라를 통해 사진을 찍는 과정으로 이해할 수 있다. [그림 7-1]과 같이 관찰지점을 정하는 것은 카메라의 위치를 선정하는 것과 같다. 관찰지점이 결정되면 이 지점에서 어떤 장면을 카메라의 사진에 담을 것인지 결정해야 한다. 즉, 어떤 방향으로 카메라를 설치할 것인지 결정한 후 사진의 구도를 정한다. 그리고 어느 방향을 사진의 위로 설정할 것인지 결정한 다음, 줌 렌즈를 통해 사진에 담을 장면의 범위를 조정한다. 마지막으로 셔터를 누르면 원하는 장면이 얻어진다. 여기에는 우리들이 미리 선택한 조리개 크기, 카메라의 렌즈 형태, 카메라 필름의 감광감도 등이 고려된 것이다.

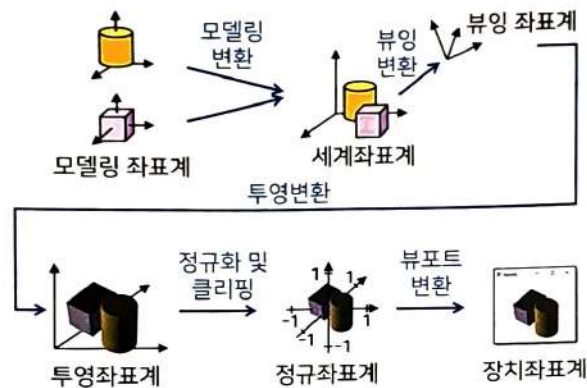


[그림 7-1] x_w - y_w - z_w 세계좌표계에서 카메라 위치와 방향

컴퓨터 그래픽스 프로그램으로 장면을 연출할 때는 카메라보다 훨씬 많은 선택사항이 있고, 좀 더 다양한 효과를 연출할 수 있기 때문에 더욱 복잡하다. 예를 들어, 3차원 입체를 2차원 평면에 나타내기 위해 평행투영 또는 원근투영을 사용할 수도 있고, 시각 방향에 따라 일부분을 선택적으로 삭제할 수도 있으며, 투영 평면을 카메라 위치

보다 더 멀리 가져갈 수도 있고, 가상의 카메라 뒤에서 물체의 영상을 얻을 수도 있다. 또한 조명 조건을 임의로 선정하여 장면을 연출할 수도 있고, 묘사된 물체 표면의 재질도 인위적으로 선택할 수 있다.

3차원 공간에서는 정해진 관찰지점과 관찰 방향에 따라 화면에 표시할 영역을 설정한다. 이렇게 설정된 공간에 대해 2차원 뷰 평면으로 투영한다. 2차원 뷰잉과 달리 클리핑 원도가 뷰 평면에 위치하고, 유한한 입체공간 영역에 대해 클리핑이 이루어진다.



[그림 7-2] 3차원 뷰잉 파이프라인

[그림 7-2]는 3차원 장면을 생성하여 최종적인 출력장치로 표시하는 일반적 처리 단계인 3차원 뷰잉 파이프라인(viewing pipeline)을 나타낸 것이다. 일단 세계좌표계에서 장면이 연출되고, 연출된 장면은 뷰잉 좌표계(viewing coordinate)로 변환된다. 뷰잉 좌표계는 뷰 평면의 위치와 방향 등의 뷰잉 파라미터를 사용하여 정의된다. 클리핑 원도는 2차원 평면으로 투영된 뷰 평면에 위치하게 된다.

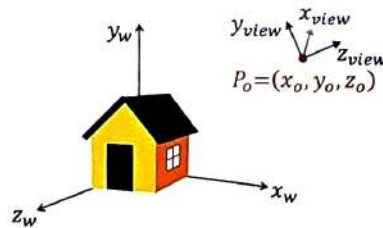
3차원 공간에 대한 클리핑 작업 영역을 뷰 체적(view volume)이라고 하며, 그 모양과 크기는 클리핑 원도, 투영의 유형, 관찰 방향에 따라 결정된다. 그리고 뷰 체적은 화면에 표시할 영역의 한계를 결정한다. 투영은 3차원 장면의 뷰잉 좌표를 뷰 평면의 2차원 좌표로 바꾸는 것을 의미한다. 물체는 정규화된 좌표로 대응되고, 뷰 체적의 외부에 있는 모든 물체는 제거된다. 클리핑 작업은 세계좌표가 정규화 좌표로 바뀐 후에 수행된다. 이 과정에서 최대의 효율을 위해 앞서 언급한 좌표변환을 모두 반영한 복합변환 행렬을 이용한다.

2차원 뷰잉과 같이 뷰포트의 앞에서 경계는 장치좌표로 표현된다. 따라서 뷰잉 알고리즘을 적용할 때 뷰포트는 장치좌표로 지정된다고 가정한다. 또한 정규화된 좌표가 클리핑 후에 뷰포트 좌표로 전달된다는 것을 가정한다. 아울러 보이는 면을 지정하고, 표면 렌더링을 위한 작업 등 추가로 필요한 작업이 있다. 마지막 단계로 선택된 출력 표시 윈도 내부에서 뷰포트 좌표를 장치좌표로 대응시킨다. 때로는 장치좌표에서 장면 연출을 할 때 스크린으로부터의 거리가 장면의 깊이 정보와 같게 되도록 윈도 좌표계로 표현되기도 한다.

7.2 3차원 뷰잉 좌표 파라미터

3차원 뷰잉 기준 프레임(viewing reference frame)을 구성하는 것은 2차원의 그것과 유사하다. 관찰자의 눈 위치에 해당되는 뷰 지점(viewing point) $P_o = (x_o, y_o, z_o)$ 을 선택하여 뷰 원점으로 사용한다. 그리고 화면에 표시될 장면의 위쪽 방향을 결정하는 뷰 업 벡터(view-up vector) V 를 선정하여 좌표축 y_{view} 를 결정한다. 다음에는 3차원 공간에서 나머지 2개의 좌표축을 결정하면 된다. 좌표축 z_{view} 의 방향은 일반적으로 관찰 방향을 의미하는 방향으로 설정한다.

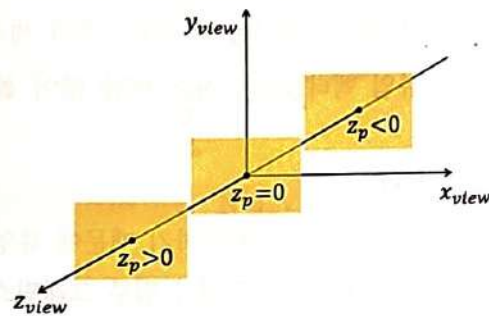
[그림 7-3]은 세계좌표에 표시된 뷰잉 좌표를 나타낸 것이다.



[그림 7-3] 오른손 좌표계를 사용한 뷰잉 좌표계의 좌표축 x_{view} , y_{view} , z_{view}

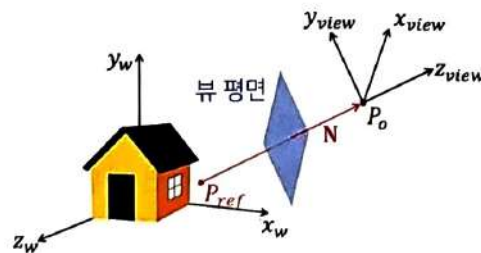
(1) 뷰 평면 법선 벡터

뷰 평면의 방향은 뷰 평면 법선 벡터(view plane normal vector) N 으로 나타낸다. 복잡도를 줄이기 위해 일반적으로 관찰 방향을 z_{view} 와 나란한 것으로 설정한다. 따라서 뷰 평면은 이 축과 수직 관계에 있다. z_{view} 축에서 뷰 평면의 위치 z_p 값에 따른 뷰 평면의 위치를 [그림 7-4]에 나타냈다. 이 z_p 값은 관찰 방향을 따라 뷰 원점으로부터 뷰 평면까지의 거리를 나타내며, 보통 음수값을 취한다. 그러므로 뷰 평면은 항상 $x_{view} - y_{view}$ 평면과 평행하고, 뷰 평면으로 물체를 투영하는 것은 연출 장면을 출력표시장치에 표시하는 첫 과정이다.



[그림 7-4] z_{view} 축을 따라 뷰 평면이 놓일 수 있는 세 가지 위치

N 의 방향을 세계좌표계의 원점으로부터 임의의 지점으로 이르는 방향으로 정의하기도 하고, 어떤 경우에는 [그림 7-5]와 같이 임의의 뷰 기준점(view reference point) P_{ref} 에서 뷰 원점 P_o 에 이르는 방향을 N 의 방향으로 택하기도 한다.



[그림 7-5] 뷰 기준점 P_{ref} 로부터 뷰잉 좌표 원점 P_o 방향의 뷰 평면 법선 벡터 N

또 다른 정의 방법으로 방향각(direction angles)을 사용하기도 한다. 여기서 방향각은 x , y , z 축과 이루는 3개의 각도를 의미하며, 각각 α , β , γ 로 나타낼 수 있다. 그러나 일반적으로 방향각을 이용하는 것보다 두 점을 이용하여 벡터의 방향을 정의하는 것을 많이 사용한다.

(2) 뷰업 벡터

뷰 평면 법선 벡터 N 이 결정되면 뷰업(view-up) 벡터 V 의 방향을 설정한다. 이 벡터는 y_{view} 의 $+$ 축 방향을 결정하는 의미가 있다. 뷰업 벡터 V 와 뷰 평면 법선 벡터 N 은 수직이어야 하지만, N 에 수직인 방향을 표현하는 것이 까다로울 수도 있다. 굳이 N 에 수직인 벡터를 지정하지 않더라도 $y_{view}-z_{view}$ 평면에만 존재하는 벡터라면 이를 이용하여 N 에 수직인 뷰업 벡터를 계산할 수 있다. 흔히 세계좌표의 y_w 와 평행하도록, 즉 $V=(0, 1, 0)$ 으로 뷰업 벡터를 설정하는 예를 많이 볼 수 있다.

(3) uvn 뷰잉 좌표 기준 프레임

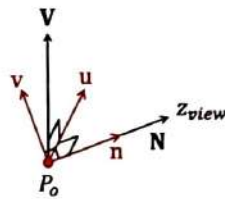
일반적으로 세계좌표가 오른손 좌표계를 사용하기 때문에 뷰잉 좌표의 기준 프레임 역시 같은 방향의 좌표계를 많이 사용한다. 그러나 일부 그래픽스 시스템의 경우 종종 왼손 좌표계의 뷰잉 좌표가 사용되기도 하는데, 이는 관찰 방향이 $+z_{view}$ 방향과 같다는 장점이 있다. 이때, z_{view} 값의 증가는 관찰 방향으로 관찰지점에서부터 멀어지는 것을 의미한다. 오른손 좌표계를 뷰잉 좌표계로 사용하는 경우 세계좌표계와 뷰잉 좌표계 모두 똑같은 오른손 좌표계를 사용하기 때문에 오직 하나의 좌표 방향만 취급하면 된다는 이점이 있다.

z_{view} 축 방향을 뷰 평면 법선 벡터 N 으로 설정하고, y_{view} 의 $+$ 축 방향(앞서 언급한 것처럼 정확한 $+y_{view}$ 방향은 아닐 수 있음)을 뷰업 벡터 V 로 결정하였기 때문에 단지 x_{view} 의 방향만 결정하면 된다. 먼저 N 을 정규화하여 z_{view} 방향의 단위 벡터 n 을 구할 수 있다. n 과 이미 정의된 벡터 V 를 통해 V 와 n 에 모두 수직인 $+x_{view}$ 축 방향의 벡터 U 를 계산할 수 있다. 오른손 좌표계를 사용하므로 벡터 U 는 V 와 n 의 외적(cross product)으로 구할 수 있으며, 이를 정규화하면 단위 벡터 u 를 구할 수 있다. 이제 y_{view} 방향의 단위 벡터 v 는 n 과 u 의 외적을 통해 구한다. V 를 N 과 수직인 벡터로 정의하지 않았더라도 이러한 절차를 통해 구한 v 는 N 과 수직이며, 정확히 y_{view}

방향을 나타낸다.

[그림 7-6]은 N 과 V 와 n, u, v 의 관계를 보여 주며, 식 (7-1)은 이와 같은 방법으로 n, u, v 를 구하는 수식이다.

$$\begin{aligned} n &= \frac{N}{|N|} = (n_x, n_y, n_z) \\ u &= \frac{V \times n}{|V \times n|} = (u_x, u_y, u_z) \\ v &= n \times u = (v_x, v_y, v_z) \end{aligned} \quad (7-1)$$



[그림 7-6] N 과 V 와 n, u, v

(4) 뷰잉 변환

벡터 u, v, n 이 결정되면 이를 이용하여 세계좌표를 뷰잉 좌표로 변환하는 뷰잉 변환을 할 수 있다. 이 과정은 5.6절에서 다른 좌표계 사이의 변환을 적용하면 되는데, 다음 두 가지 변환을 차례로 하는 것으로 볼 수 있다.

- ① 뷰 원점을 세계좌표 원점으로 이동하는 이동변환
- ② 뷰잉 좌표축 $x_{view}, y_{view}, z_{view}$ 가 세계좌표축 x_w, y_w, z_w 와 일치하도록 하는 회전변환

뷰 원점 $P_o = (x_o, y_o, z_o)$ 를 세계좌표 원점으로 이동시키는 변환행렬은 다음과 같이 정의할 수 있다.

$$T_{VT} = \begin{bmatrix} 1 & 0 & 0 & -x_o \\ 0 & 1 & 0 & -y_o \\ 0 & 0 & 1 & -z_o \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7-2)$$

또한 세계좌표계에서 뷰잉 좌표축 $x_{view}, y_{view}, z_{view}$ 방향의 단위 벡터 u, v, n 이 식 (7-1)과 같이 정의된다면 $x_{view}, y_{view}, z_{view}$ 축을 x_w, y_w, z_w 축으로 회전하는 것은 다음과 같이 정의할 수 있다.

$$R_{VT} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7-3)$$

그러므로 뷰잉 변환행렬 M_{VT} 는 다음과 같다.

$$\begin{aligned} M_{VT} &= R_{VT} T_{VT} \\ &= \begin{bmatrix} u_x & u_y & u_z & -(x_o u_x + y_o u_y + z_o u_z) \\ v_x & v_y & v_z & -(x_o v_x + y_o v_y + z_o v_z) \\ n_x & n_y & n_z & -(x_o n_x + y_o n_y + z_o n_z) \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{P}_o \\ v_x & v_y & v_z & -\mathbf{v} \cdot \mathbf{P}_o \\ n_x & n_y & n_z & -\mathbf{n} \cdot \mathbf{P}_o \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned} \quad (7-4)$$

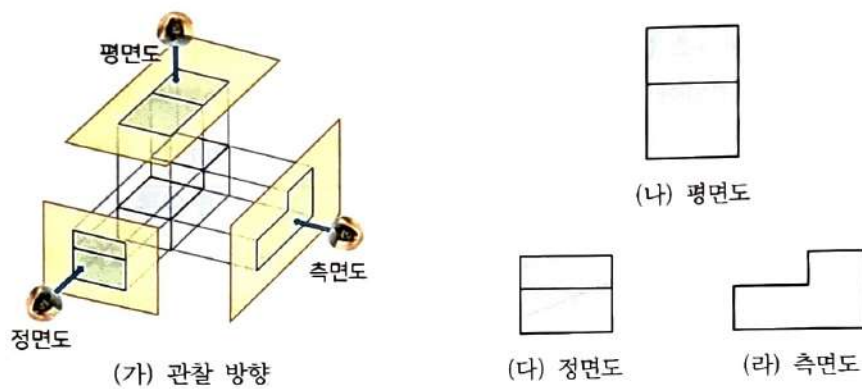
7.3 평행투영

평행투영(parallel projection)이란 좌표점을 뷰 평면으로 이동시킬 때, 평행한 투영선을 따라 이동시키는 기법이다. 평행투영에서는 물체의 비례관계가 그대로 유지되고, 3차원 물체의 외형을 나타내는 CAD(Computer-Aided Design)에서 많이 사용된다. 장면 안의 모든 평행선은 평행투영을 사용했을 때 뷰 평면에서도 모두 평행하게 표시된다. 평행투영기법으로 물체를 뷰 평면으로 투영할 때, 뷰 평면과 투영선의 관계에 따라 크게 두 가지 방법이 있다. 즉, 뷰 평면에 수직인 평행선을 따라 투영하는 직교투영과 평행한 사선을 따라 물체를 뷰 평면에 투영시키는 경사투영으로 구분된다.

7.3.1. 직교투영

(1) 개요

물체를 뷰 평면과 수직인 투영선을 따라 뷰 평면으로 투영하는 것을 직교투영(orthogonal projection)이라고 한다. 공학이나 건축 등의 도면에서는 물체의 길이나 각도 등을 정확하게 표현하기 위해 [그림 7-7]과 같이 물체의 앞과 옆, 위에서 바라본 그림을 많이 사용하는데, 이를 각각 정면도, 측면도, 평면도라고 한다. 이러한 도면에서는 뷰 평면이 x , y , z 축과 수직이다. 따라서 하나의 축과 교차한다.



[그림 7-7] 직교투영: 정면도, 측면도, 평면도

한 도면에 물체의 여러 방향의 면을 표현하면 물체의 전체적인 모양을 이해하는 데 도움이 된다. 이를 위해서는 [그림 7-8]의 (가)와 같이 뷰 평면이 여러 축과 교차하도록



[그림 7-8] 축각직교투영

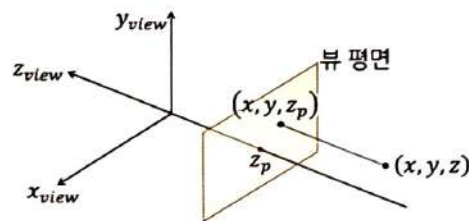
록 직교투영을 하는데, 이를 축각직교투영(axonometric orthogonal projection)이라고 한다. 특히 물체가 정의된 주축의 원점으로부터 동일한 거리에서 세 좌표축과 교차하도록 할 때, 이를 등각투영(isometric projection)이라고 한다. 등각투영에서는 x , y , z 축 방향의 비율이 동일하며, 이 세 축을 투영하면 이들 사이의 각도는 모두 120도가 된다.

(2) 직교투영 좌표

[그림 7-9]와 같이 투영 방향이 z_{view} 와 평행한 경우, 임의의 좌표 (x, y, z) 는 간단한 변환 방정식으로 투영좌표를 다음과 같이 구할 수 있다.

$$x_p = x, \quad y_p = y \quad (7-5)$$

투영변환에 대한 z 축 값은 가시성을 결정하기 위한 정보로 사용된다. 그리고 3차원 좌표는 정규화된 공간의 위치좌표로 바뀌게 된다.



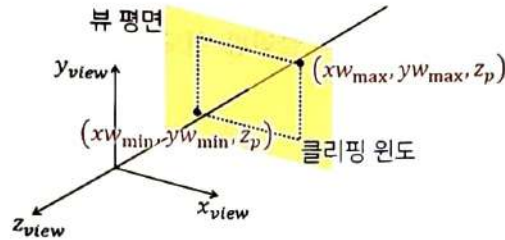
[그림 7-9] 임의의 좌표의 뷰 평면에 대한 직교투영

(3) 클리핑 원도와 직교투영의 뷰 체적

세계좌표계에 정의되는 클리핑 원도를 사용하는 2차원 뷰잉과는 달리, 3차원 뷰잉은 뷰 평면 위에 좌측 하단과 우측 상단의 2차원 좌표지점을 지정하여 클리핑 원도를 설정한다. 따라서 그 원도는 [그림 7-10]과 같이 x_{view} , y_{view} 축과 평행한 사각형이다. 만약 다른 형태의 클리핑 원도나 방향을 원한다면, 그 목적에 맞는 뷰잉 함수를 만들어야 한다.

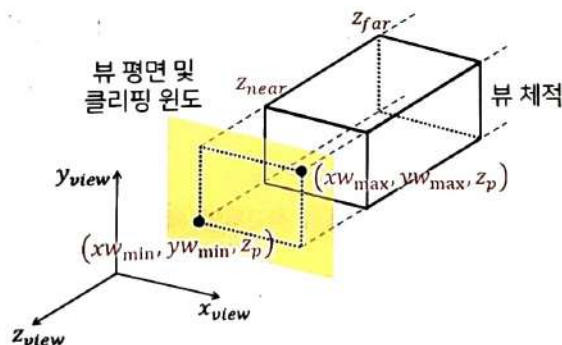
3차원 공간의 내용이 뷰 평면에 투영되고, 뷰 평면에서 클리핑 원도가 정의되었기 때문에 클리핑 원도의 모서리는 화면에 표시하기 원하는 범위의 x 와 y 의 경계를 지정한다. 따라서 뷰 체적은 3차원 공간에서 클리핑 영역의 위, 아래, 양 옆의 경계에 의해

영역이 정의되며, 이 4개의 경계는 뷰 평면과 수직인 평면이다.



[그림 7-10] 뷰 평면상의 클리핑 윈도우

z_{view} 방향으로 뷰 체적의 범위는 뷰 평면과 평행한 2개의 평면으로 지정된다. 먼 평면(far plane)은 뷰 평면에 투영될 장면의 뒤 공간의 한계를, 가까운 평면(near plane)은 앞 공간의 한계를 나타낸다. 일반적으로 관찰 방향을 z_{view} 방향과 반대로 설정하기 때문에 z_{view} 축에서 본다면 $z_{far} < z_{near}$ 관계가 성립한다. 가까운 평면과 먼 평면이 지정 되면 유한한 직교 뷰 체적을 [그림 7-11]과 같이 얻을 수 있으며, 그 형태는 직육면체이다. 뷰 체적 내부의 물체만 표시하고, 나머지 밖의 부분은 클리핑 알고리즘을 통해 제거한다.



[그림 7-11] 가까운 평면과 먼 평면에 의한 직교투영 뷰 체적

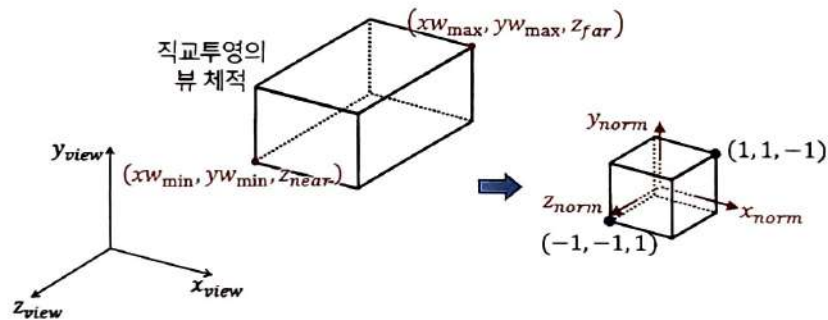
일반적으로 먼 평면과 가까운 평면은 다양한 효과를 연출하기 위해 임의의 위치를 취할 수도 있고, 심지어 서로 뷰 평면의 반대에 위치할 수도 있다. 마찬가지로 뷰 평면

의 위치도 먼 클리핑 평면과 가까운 클리핑 평면과의 관계를 고려할 때 어떤 곳에도 위치할 수 있다. 클리핑 평면과 뷰 평면에 대해 다양한 선택이 가능하지만, 처리 작업의 복잡도를 줄이고 효율적인 처리를 위해 대부분의 경우 가까운 클리핑 평면과 뷰 평면을 같은 위치에 설정하고, 사각형의 클리핑 윈도를 사용한다.

(4) 직교투영을 위한 정규화 변환

뷰 체적에 대한 경계가 설정되면 이 직육면체 안의 모든 좌표는 뷰 평면으로 투영된다. 여기서 직교투영을 통해 뷰 평면으로 좌표점을 옮기게 되므로 3차원 공간의 한 점 (x, y, z) 에 대한 투영점은 (x, y) 가 된다. 정규화된 뷰 체적으로 단위 정육면체를 사용하는 경우 x, y, z 축의 범위를 0과 1 사이로 설정한다. 또는 정규화된 좌표축의 범위를 -1 부터 1 까지의 정육면체를 사용하기도 한다.

가까운 평면과 먼 평면의 z 좌표를 각각 z_{near} 와 z_{far} 라고 표시한다. 따라서 [그림 7-12]와 같이 좌표 $(x_{min}, y_{min}, z_{near})$ 는 $(-1, -1, 1)$ 로 대응되고, $(x_{max}, y_{max}, z_{far})$ 는 $(1, 1, -1)$ 로 대응된다.



[그림 7-12] 오른손 좌표계에서 직교투영 뷰 체적의 정규화 변환

직육면체 뷰 체적을 정규화된 정육면체로 변환하는 것은 전체적으로 2단계로 나누어 생각할 수 있다. 첫째, 뷰 체적의 중심을 정규화된 정육면체의 중심, 즉 원점으로 이동하는 과정으로서 이는 이동변환을 의미한다. 둘째, 두 입체 체적의 모서리 크기를 일치시키는 과정으로서 이는 크기변환을 의미한다. 6.2절에서 설명한 클리핑 윈도를 정규화된 정사각형으로 변환하는 방법과 유사하다. 변환행렬은 식 (6-4)에 z 축을 추가

하여 확장한 것과 같다. 이때, z_{far} 로부터 z_{near} 의 범위가 -1과 1 사이로 대응된다. 따라서 직교 뷰 체적에 대한 정규화 변환은 식 (7-6)과 같다.

$$M_{ot} = \begin{bmatrix} \frac{2}{xw_{max} - xw_{min}} & 0 & 0 & -\frac{xw_{max} + xw_{min}}{xw_{max} - xw_{min}} \\ 0 & \frac{2}{yw_{max} - yw_{min}} & 0 & -\frac{yw_{max} + yw_{min}}{yw_{max} - yw_{min}} \\ 0 & 0 & \frac{2}{z_{near} - z_{far}} & -\frac{z_{near} + z_{far}}{z_{near} - z_{far}} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7-6)$$

만약 스크린 좌표계가 왼손 좌표계를 사용한다면 관찰 방향의 거리가 뷰 평면으로부터의 거리로 직접 해석된다는 이점이 있다. 이 경우 투영좌표가 왼손 좌표계로 표현된 정규화 좌표로 바뀌어야 한다. 따라서 투영좌표는 뷰포트 변환에 의해 왼손 좌표계로 표현된 관찰좌표로 변환된다. 이 과정은 크기변환의 $S(1, 1, -1)$ 을 통해 구할 수 있다. 또한 z_{near} 와 z_{far} 를 왼손 좌표계의 값으로 지정하였다면 z_{near} 와 z_{far} 를 $-z_{near}$ 와 $-z_{far}$ 로 변경하면 된다. 따라서 왼손 좌표계를 사용할 경우의 정규화 변환행렬은 식 (7-7)과 같다.

$$M_{ot, LH} = \begin{bmatrix} \frac{2}{xw_{max} - xw_{min}} & 0 & 0 & -\frac{xw_{max} + xw_{min}}{xw_{max} - xw_{min}} \\ 0 & \frac{2}{yw_{max} - yw_{min}} & 0 & -\frac{yw_{max} + yw_{min}}{yw_{max} - yw_{min}} \\ 0 & 0 & \frac{2}{z_{near} - z_{far}} & -\frac{z_{near} + z_{far}}{z_{near} - z_{far}} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7-7)$$

이 단계의 뷰잉 파이프라인에서 장치와 무관한 좌표변환이 완성되고 하나의 복합변환 행렬로 도출된다. 따라서 클리핑 과정은 정규화 변환 후에 수행된다. 클리핑 작업 후에 가시성 검사, 표면 렌더링 등의 작업이 수행되어 최종적인 장면이 스크린에 표출된다.

7.3.2. 경사평행투영

평행투영은 물체의 좌표를 뷰 평면의 법선 벡터와 연관된 투영 경로를 따라 뷰 평면에 투영한다. 투영 경로가 뷰 평면과 수직이 아닌 경우를 경사평행투영(oblique parallel

projection) 또는 경사투영이라고 한다. 이 투영기법을 사용하면 [그림 7-13]과 같이 정면도, 측면도, 평면도를 하나의 화면에 함께 표시할 수 있는 장점이 있다. 경사투영은 투영 경로의 방향을 여러 가지 방법으로 지정할 수 있다.

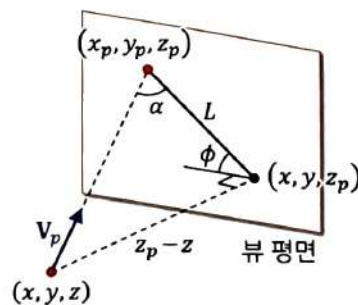


[그림 7-13] 정육면체의 경사투영

(1) 경사투영의 해석

공학과 건축 분야에 많이 응용되는 경사투영은 [그림 7-14]에 나타난 것과 같이 2개의 각도로 정의된다. 3차원 공간의 좌표 (x, y, z) 는 뷰 평면의 (x_p, y_p, z_p) 로 투영된다. 이 투영점은 z_{view} 좌표축의 z_p 지점에 위치한다. 여기서 점 (x, y, z) 를 뷰 평면에 수직으로 내린 점 (x, y, z_p) 와 점 (x_p, y_p, z_p) 를 잇는 직선 및 x_{view} 축이 이루는 각도를 ϕ 라 하면, 식 (7-8)과 같은 관계식을 얻을 수 있다.

$$\begin{aligned} x_p &= x + L \cos \phi \\ y_p &= y + L \sin \phi \end{aligned} \quad (7-8)$$



[그림 7-14] 경사투영

이때, 점 (x, y, z) 와 (x, y, z_p) 를 잇는 선분은 점 (x, y, z_p) 와 (x_p, y_p, z_p) 를 잇는 길이 L 의 선분으로 투영되는데, (x, y, z) 에서 (x_p, y_p, z_p) 로 향하는 경사투영의 경로선과 뷰 평면 사이의 각도를 α 라 하면 L 은 다음과 같다.

$$L = \frac{z_p - z}{\tan \alpha} \quad (7-9)$$

만일 α 가 45도이면 $\tan \alpha = 1$ 이므로 L 은 $z_p - z$ 와 같고, $\tan \alpha = 2$ 인 α (약 63.4도)를 선택하면 L 은 $z_p - z$ 의 반이다. $\tan \alpha = 1$ 인 경우 캐버리어 투영(cavalier projection)이라 하며, 정육면체를 캐버리어 투영하면 세 변의 길이가 그대로 유지된다. 반면에, $\tan \alpha = 2$ 인 경우 캐비닛 투영(cabinet projection)이라 하며, 정육면체를 캐비닛 투영하면 뷰 평면에 평행한 변은 길이가 그대로 유지되고, 뷰 평면에 수직인 변은 원래 길이의 반으로 표시된다. [그림 7-15]를 통해 캐버리어 투영과 캐비닛 투영을 비교해 볼 수 있다.



[그림 7-15] 정육면체의 경사투영

(2) 경사투영 벡터

그래픽스 라이브러리에서 경사투영의 투영 방향은 뷰 평면을 향한 투영 벡터 $\mathbf{V}_p$ 를 이용하여 정의한다. 이 방향 벡터는 뷰 평면의 법선 벡터를 정의하기 위해 지정한 뷰 기준점처럼 $\mathbf{V}_p$ 방향으로 뷰 지점을 향하는 경사투영 기준점을 지정하여 정의한다. 투영 벡터 $\mathbf{V}_p = (v_{px}, v_{py}, v_{pz})$ 가 설정되면 3차원 공간의 모든 좌표는 이 벡터와 평행한 투영선을 따라 뷰 평면으로 이동한다. 따라서 임의의 한 점 (x, y, z) 가 투영 벡터 $\mathbf{V}_p$ 를 따라 뷰 평면에 투영된 좌표를 (x_p, y_p, z_p) 라고 한다면, [그림 7-14]에서와 같이 식 (7-10)의 관계식이 성립한다.

$$\begin{aligned}\frac{x_p - x}{z_p - z} &= \frac{v_{px}}{v_{pz}} \\ \frac{y_p - y}{z_p - z} &= \frac{v_{py}}{v_{pz}}\end{aligned}\tag{7-10}$$

그러므로 경사투영에 의해 점 (x, y, z) 가 뷰 평면에 투영되는 좌표 (x_p, y_p) 는 식 (7-11)과 같다.

$$\begin{aligned}x_p &= x + (z_p - z) \frac{v_{px}}{v_{pz}} \\ y_p &= y + (z_p - z) \frac{v_{py}}{v_{pz}}\end{aligned}\tag{7-11}$$

식 (7-11)에서 $v_{px} = v_{py} = 0$ 일 경우 직교투영이 된다.

(3) 경사투영의 뷰 체적

뷰 평면 위의 좌표점 (xw_{min}, yw_{min}) 과 (xw_{max}, yw_{max}) 으로 직사각형의 클리핑 원도를 설정한다. 투영 벡터와 클리핑 원도 모서리에 따라 뷰 체적의 상·하·좌·우가 결정된다. 추가로 가까운 평면과 먼 평면을 지정하여 뷰 체적의 범위를 결정한다. 이와 같은 경사투영의 뷰 체적은 평행육면체 모양이다. 경사투영은 투영 방향이 정의되는 방법에 따라 뷰 평면의 위치가 바뀌게 된다. 따라서 투영 벡터의 기준점을 고정시킨 상태에서 뷰 평면 또는 클리핑 원도 위치를 변경하면 뷰 체적의 형체가 변화하게 된다.

(4) 경사투영 변환행렬

투영 벡터로 표현된 투영관계식인 식 (7-11)을 변환행렬로 나타내면 식 (7-12)와 같다.

$$M_{ob} = \begin{bmatrix} 1 & 0 & -\frac{v_{px}}{v_{pz}} & z_p \frac{v_{px}}{v_{pz}} \\ 0 & 1 & -\frac{v_{py}}{v_{pz}} & z_p \frac{v_{py}}{v_{pz}} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\tag{7-12}$$

이 행렬을 통해 x, y 좌표는 뷰 평면과의 거리, 즉 z_{view} 축의 z_p 지점까지의 거리에 비례하여 이동변환한다. 공간에서 z 축의 값은 바뀌지 않고 그대로 유지된다. 만약 $v_{px} = v_{py} = 0$ 이라면 식 (7-12)는 단위행렬이 되어 단순한 직교투영이 얻어진다.

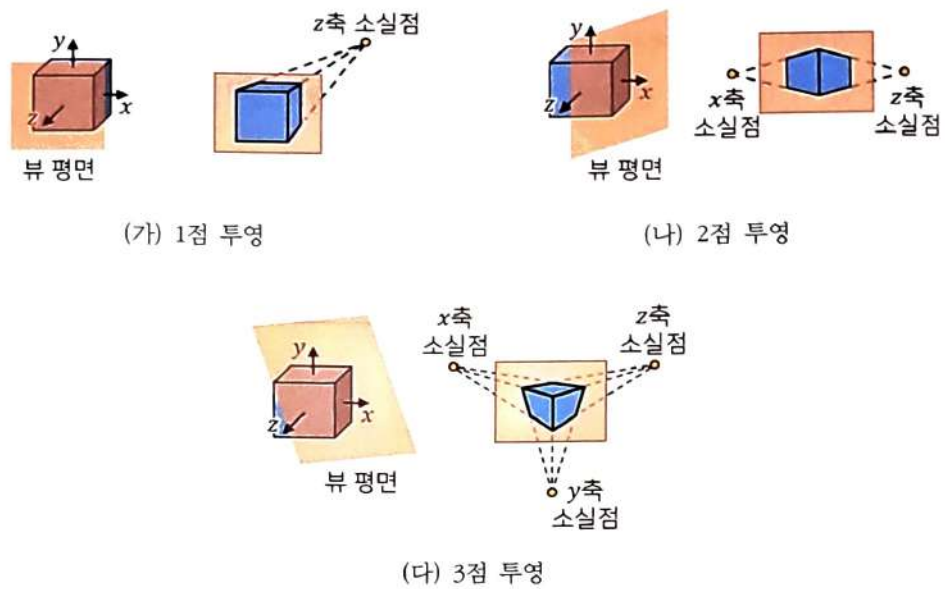
7.4 원근투영

7.4.1. 개요

평행투영(직교투영, 경사투영)이 간단하고 물체의 비례관계가 그대로 유지된다는 장점이 있지만, 현실감 있는 장면을 연출하기에는 많은 한계가 있다. 따라서 카메라 사진과 같은 입체감을 표현하기 위해 물체의 모습이 필름에 모아지는 과정을 적용해야 한다. 원근투영(perspective projection)은 기하광학적 효과를 통해 투영기준점(projection reference point)으로 모아지는 투영선을 따라 물체를 뷰 평면으로 투영하는 방법이다. 따라서 같은 크기의 물체에 대해 뷰 평면으로부터 먼 거리에 있는 물체가 가까운 거리의 물체보다 작게 표시된다.

서로 평행한 선분들이 있을 때, 만일 이들이 뷰 평면에 평행하다면 원근투영 결과 서로 평행한 선분으로 투영된다. 반면에, 이 선분들이 뷰 평면과 평행하지 않다면 이 선분들은 평행하지 않은 선분으로 투영되며, 이들의 연장선은 어느 점으로 모이게 된다. 이때, 평행한 직선이 한곳으로 모이는 점을 소실점(vanishing point)이라고 한다.

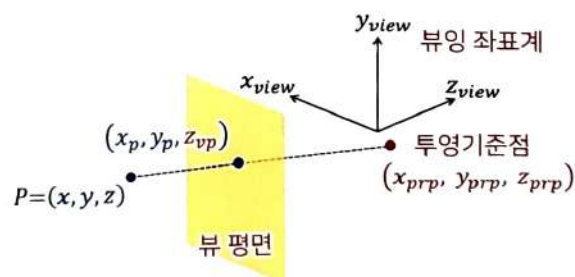
물체의 주축과 평행한 직선군에 대한 소실점을 주소실점이라고 한다. 뷰 평면의 방향에 따라 주소실점이 1개, 2개, 3개가 될 수 있는데, 이는 뷰 평면이 교차하는 주축의 수와 같다. 각각의 경우를 1점 투영, 2점 투영, 3점 투영이라고 한다. [그림 7-16]은 주소실점 수에 따른 투영의 형태를 보여 준다.



[그림 7-16] 원근투영의 유형

7.4.2. 원근투영의 좌표변환

원근투영에서 공간좌표 (x, y, z) 가 투영기준점 $(x_{prp}, y_{prp}, z_{prp})$ 를 통해 투영되는 과정을 [그림 7-17]에 나타냈다. 투영선과 뷰 평면의 교점은 (x_p, y_p, z_{vp}) 가 되고, z_{vp} 는 z_{view} 축에서 뷰 평면 위치이다. 이때, 원근투영선을 매개방정식으로 표현하면 다음과 같다.



[그림 7-17] 좌표 (x, y, z) 의 점 P 가 투영기준점에 대하여 원근투영되는 모습

$$\begin{aligned}
x' &= x + (x_{prp} - x)u \\
y' &= y + (y_{prp} - y)u \quad 0 \leq u \leq 1 \\
z' &= z + (z_{prp} - z)u
\end{aligned} \tag{7-13}$$

이 투영선상의 임의의 점이 (x', y', z') 이라고 한다면 $u=0$ 인 경우에 $P=(x, y, z)$ 를 의미하고, $u=1$ 인 경우에는 이 직선의 다른 한쪽 끝인 투영기준점의 좌표 $(x_{prp}, y_{prp}, z_{prp})$ 가 된다. 뷰 평면의 위치는 $z' = z_{vp}$ 이므로 뷰 평면에서 매개변수 u 에 대하여 z' 방정식을 풀면 식 (7-14)와 같다.

$$u = \frac{z_{vp} - z}{z_{prp} - z} \tag{7-14}$$

u 의 값을 식 (7-13)에 대입하면 일반적인 원근투영 변환식이 만들어진다.

$$\begin{aligned}
x_p &= x \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right) + x_{prp} \left(\frac{z_{vp} - z}{z_{prp} - z} \right) \\
y_p &= y \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right) + y_{prp} \left(\frac{z_{vp} - z}{z_{prp} - z} \right)
\end{aligned} \tag{7-15}$$

원근투영에서 투영점을 계산하는 것은 평행투영보다 복잡하다. 따라서 원근투영의 과정을 특수한 경우로 제한하여 좀 더 간소화시킨 변환식을 사용하기도 한다. 예를 들어, 투영기준점이 좌표축의 원점에 위치한다고 가정한다면 $x_{prp} = y_{prp} = z_{prp} = 0$ 이므로 식 (7-15)는 식 (7-16)과 같이 간소화된다.

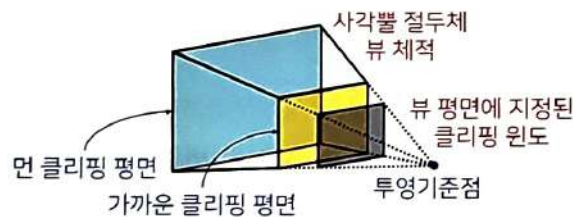
$$\begin{aligned}
x_p &= x \left(\frac{z_{vp}}{z} \right) \\
y_p &= y \left(\frac{z_{vp}}{z} \right)
\end{aligned} \tag{7-16}$$

투영기준점을 설정할 때는 몇 가지 현상을 고려해야 한다. 만약 투영기준점이 뷰 평면 위에 위치한다면 전체 영상이 한 점으로 투영된다. 또한 투영기준점이 관찰 방향의 앞쪽에 위치한다면 투영기준점 앞쪽의 영상은 뷰 평면에 거꾸로 투영되며, 뷰 평면과 투영기준점 사이의 장면은 단순히 확대되어 뷰 평면에 투영된다. 따라서 관찰 방향을 고려할 때 장면과 투영기준점 사이에 뷰 평면이 위치하는 것이 일반적이다.

만약 투영기준점이 뷰 평면과 가깝다면 원근효과는 강조된다. 즉, 같은 크기의 물체에 대하여 가까운 물체가 먼 곳의 물체보다 더욱 크게 보인다. 마찬가지로 투영기준점이 뷰 평면에서 멀어지면 가까운 물체와 먼 곳의 물체 크기 차이가 줄어든다. 투영기준점이 뷰 평면으로부터 아주 멀리 떨어지게 되면 평행투영과 유사한 효과를 보인다.

7.4.3. 원근투영의 뷰 체적

뷰 평면에 클리핑 윈도를 지정하면 뷰 체적이 만들어진다. 투영선이 평행하지 않기 때문에 뷰 체적을 위한 경계 역시 평행하지 않다. 뷰 체적의 위와 아래, 양옆은 평면이고, 내부의 모든 점은 투영기준점을 통과한다. 따라서 [그림 7-18]과 같이 원근투영의 뷰 체적은 꼭짓점이 투영기준점인 사각뿔 모양이다. 이 사각뿔 외부의 모든 물체는 클리핑 작업을 통해 제거된다. z_{view} 축과 수직인 가까운 클리핑 평면과 먼 클리핑 평면을 추가하여 잘려진 사각뿔 모양의 절두체(frustum)가 된다.



[그림 7-18] 일반적인 원근투영의 뷰 체적

일반적으로 가까운 클리핑 평면과 먼 클리핑 평면은 투영기준점에 대하여 같은 방향에 놓이고, 먼 평면이 가까운 평면보다 투영기준점에서 멀리 놓인다. 그리고 먼 평면과 가까운 평면을 통해 뷰 평면에 투영되는 범위를 제한한다. 특히 투영기준점에 아주 가까이 위치한 물체는 알아볼 수 없을 정도로 크게 투영되므로, 가까운 클리핑 평면은 이러한 물체를 제거하는 역할을 한다. 또한 투영기준점에서 너무 멀리 떨어진 작은 물체는 알아볼 수 없을 정도로 작은 점으로 투영되므로, 먼 클리핑 평면은 이러한 물체를 제거하는 역할을 한다.

원근투영변환은 [그림 7-18]의 사각뿔 절두체 뷰 체적을 직육면체로 만들며, 정규화

변환을 통해 각 좌표축의 범위가 -1부터 1 사이인 정육면체가 되게 만든다.

7.4.4. 원근투영변환행렬

식 (7-7)과 (7-12)의 변환행렬을 통해 알 수 있는 것처럼 평행투영은 아핀 변환이다. 그러나 원근투영은 투영좌표를 구하는 식 (7-15)의 분모에 z 성분값이 있는데, 이것은 아핀 변환행렬로 표현할 수 없다. 식 (7-15)의 분모가 원근투영된 동차좌표의 동차 파라미터가 되도록 변환행렬을 만들면 원근투영변환을 실현할 수 있다. 식 (7-15)를 식 (7-17)의 형태로 표현해 보자.

$$\begin{aligned}x_p &= \frac{x_h}{h} \\y_p &= \frac{y_h}{h}\end{aligned}\tag{7-17}$$

여기서 동차 파라미터 h 는 식 (7-18)과 같다.

$$h = z_{prp} - z\tag{7-18}$$

또한 x_h 와 y_h 는 식 (7-19)와 같다.

$$\begin{aligned}x_h &= x(z_{prp} - z_{vp}) - x_{prp}z + x_{prp}z_{vp} \\y_h &= y(z_{prp} - z_{vp}) - y_{prp}z + y_{prp}z_{vp}\end{aligned}\tag{7-19}$$

3차원 공간의 데카르트 좌표 (x, y, z) 는 $(x, y, z, 1)$ 이라는 동차좌표로 표현한다. 만일 (x, y, z) 를 동차 파라미터가 $h \neq 0$ 인 동차좌표로 표현한다면 (hx, hy, hz, h) 가 된다. 따라서 동차좌표로 나타낸 원근투영변환 결과가 (x_h, y_h, z_h, h) 라면 구하고자 하는 투영좌표는 식 (7-20)과 같다.

$$(x_p, y_p, z_p) = \left(\frac{x_h}{h}, \frac{y_h}{h}, \frac{z_h}{h} \right)\tag{7-20}$$

즉, 원근투영변환된 좌표를 얻기 위해 각 좌표 성분을 동차 파라미터 h 로 나누는 과정이 필요하다. 그런데 좌표를 동차 파라미터로 나누는 것은 z 좌표값을 왜곡할 수 있다. 원근투영변환에 따라 계산한 x_h 와 y_h 를 깊이(z 값)에 비례한 크기변환을 하여 x_p 와

y_p 를 구해야 하지만, z 축의 좌표는 깊이와 무관해야 한다. 따라서 깊이 정보를 보존하면서 동차좌표 x_h 와 y_h 를 계산할 수 있는 행렬을 구성해야 한다. 이러한 특성과 식 (7-18)과 (7-19)를 만족하는 원근투영행렬은 식 (7-21)과 같이 구성할 수 있다.

$$M_{pp} = \begin{bmatrix} z_{prp} - z_{vp} & 0 & -x_{prp} & x_{prp} z_{vp} \\ 0 & z_{prp} - z_{vp} & -y_{prp} & y_{prp} z_{vp} \\ 0 & 0 & s_z & t_z \\ 0 & 0 & -1 & z_{prp} \end{bmatrix} \quad (7-21)$$

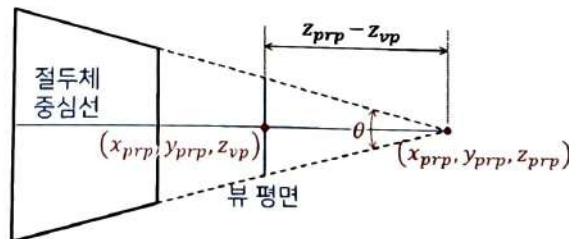
행렬요소 s_z 와 t_z 는 투영된 z 값을 정규화하기 위한 크기변환과 이동변환 요소이다. s_z 와 t_z 값은 z 축의 좌표가 원하는 범위가 될 수 있게 정한다. 7.4.6항에서 z 축 좌표를 정규화하는 s_z 와 t_z 를 구하는 과정을 살펴볼 것이다.

적절한 원근투영 조건을 설정하여 식 (7-21)을 간소화할 수 있다. 예를 들어, 투영기준점이 좌표축의 원점에 위치한다고 가정하면 $x_{prp} = y_{prp} = z_{prp} = 0$ 이므로 식 (7-21)은 식 (7-22)와 같이 간소화된다.

$$M_{pp} = \begin{bmatrix} -z_{vp} & 0 & 0 & 0 \\ 0 & -z_{vp} & 0 & 0 \\ 0 & 0 & s_z & t_z \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad (7-22)$$

7.4.5. 원근투영 절두체

원근투영 절두체의 중심선은 뷰 체적을 통과하는 직선으로 투영기준점과 클리핑 원도의 중심을 지난다. [그림 7-19]와 같은 대칭형의 절두체는 그 중심선이 뷰 평면과 수



[그림 7-19] 대칭형 절두체인 원근투영의 뷰 체적

직인 경우를 의미한다. 절두체의 중심선과 뷰 평면의 교점이 $(x_{prp}, y_{prp}, z_{vp})$ 이므로 클리핑 원도의 꼭짓점 위치를 원도 크기로 나타내면 식 (7-23)과 같다($width$ 와 $height$ 는 클리핑 원도의 폭과 높이임).

$$\begin{aligned} xw_{\min} &= x_{prp} - \frac{width}{2}, & xw_{\max} &= x_{prp} + \frac{width}{2} \\ yw_{\min} &= y_{prp} - \frac{height}{2}, & yw_{\max} &= y_{prp} + \frac{height}{2} \end{aligned} \quad (7-23)$$

따라서 클리핑 원도의 좌표 대신에 원도의 폭과 높이를 이용하여 대칭형 원근투영의 요솟값을 정의할 수 있다. 사진의 경우 대칭형의 원근투영기법으로 장면을 필름 평면에 투영시킨다. 즉, 장면에 있는 물체로부터 반사된 빛이 카메라의 '시각원추(cone of vision)'를 통해 필름에 모아진다. 이때, 시각원추는 카메라 렌즈 크기의 단위로 알려진 가시각(field-of-view angle)이라고도 한다. 예를 들어, 큰 가시각은 광각 렌즈에 해당된다. 컴퓨터 그래픽스에서 시각원추는 대칭형 절두체로 대신하며, 가시각을 절두체의 각도로 해석한다.

일반적으로 가시각은 [그림 7-19]와 같이 클리핑 원도의 위 평면과 아래 평면이 이루는 각도를 의미하므로, 클리핑 원도의 높이는 결정되나 폭은 결정되지 않는다. 따라서 완전한 클리핑 원도 규격을 결정하기 위해 원도 폭과 종횡비(aspect ratio)를 사용하기도 한다. 종횡비 r_{asp} 는 식 (7-24)와 같이 정의한다.

$$r_{asp} = \frac{width}{height} \quad (7-24)$$

[그림 7-19]의 삼각형에서 다음의 관계를 확인할 수 있다.

$$\tan\left(\frac{\theta}{2}\right) = \frac{height/2}{z_{prp} - z_{vp}} \quad (7-25)$$

따라서 클리핑 원도의 높이는 다음과 같이 계산된다.

$$height = 2(z_{prp} - z_{vp}) \tan\left(\frac{\theta}{2}\right) \quad (7-26)$$

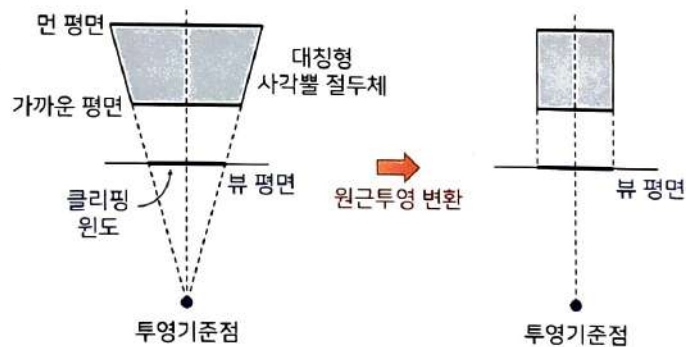
식 (7-21)의 행렬요소의 값은 식 (7-27)의 식으로 바꿀 수 있다.

$$z_{prp} - z_{vp} = \frac{height}{2} \cot\left(\frac{\theta}{2}\right) = \frac{width \cdot \cot(\theta/2)}{2 \cdot r_{asp}} \quad (7-27)$$

어떤 경우에는 투영기준점과 뷰 평면을 기준으로 원근투영을 정의하기도 한다. 이 경우 가시각, 클리핑 원도의 종횡비, 관찰지점에서 가까운 클리핑 원도 평면과 먼 클리핑 원도 평면의 거리를 미리 설정하여 대칭형의 원근투영을 완전히 정의하는 것이다. 여기서 사용된 종횡비가 일반적으로 뷰포트 과정에도 적용한다.

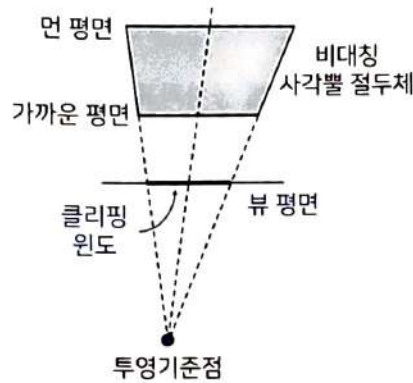
가시각을 축소하면 원근투영에서 원근효과가 축소되는 결과를 가져온다. 이는 뷰 평면으로부터 투영기준점을 멀리 이동시킨 것과 같다. 또한 가시각이 축소되면 클리핑 원도의 높이가 상대적으로 줄어들고, 작은 영역을 zoom인하는 효과를 제공한다. 같은 원리로 큰 가시각은 결국 클리핑 원도를 높게 만들고 그 결과 원근효과가 증대되는데, 이는 마치 투영기준점과 뷰 평면이 가까이 있는 경우와 같다.

원근투영에서 대칭형의 절두체로 뷰 체적이 형성될 때, 원근변환은 절두체 내부의 좌표를 직육면체의 직교투영좌표로 대응된다. [그림 7-20]과 같이 절두체의 중심선은 뷰 평면에 대하여 수직이고, 직육면체의 중심선과 일치한다. 따라서 절두체에서 같은 투영선 위의 모든 점은 뷰 평면의 동일한 점 (x_p, y_p) 로 대응되며, 뷰 체적의 모든 투영선은 원근변환에 의해 뷰 평면과 수직인 직선으로 바뀌고 절두체의 중심선과 평행해진다.



[그림 7-20] 대칭형의 절두체 뷰 체적과 원근투영변환에 의한 직육면체

원근투영의 뷰 체적 중심선이 뷰 평면에 대하여 수직이 아니면 [그림 7-21]과 같은 비대칭형 절두체를 형성한다. 이런 경우에는 뷰 체적을 먼저 대칭형 절두체로 변환시



[그림 7-21] 뷰 평면이 투영기준점과 가까운 클리핑 평면 사이에 있는 경우의 비대칭형 절두체

킨 다음, 정규화 뷰 체적으로 변환시켜야 한다. 비대칭형 원근투영 뷰 체적은 z 축 밀림 변환을 적용하여 대칭형의 절두체로 변환될 수 있다. 이 변환은 z 축 기준점으로부터 거리에 비례하는 만큼 옆으로 이동시키는 변환이다. 이 경우 기준점은 투영기준점의 z 축 좌표 z_{pp} 이다. 그리고 클리핑 원도의 중심을 뷰 평면상의 (x_{pp}, y_{pp}) 로 이동시켜야 한다. 이 변환과정을 단순화하기 위해 투영기준점을 뷰잉 좌표의 원점이 되도록 설정한다. 투영기준점을 $(x_{pp}, y_{pp}, z_{pp}) = (0, 0, 0)$ 으로 설정했다면 필요한 밀림변환행렬은 식 (7-28)과 같다.

$$H = \begin{bmatrix} 1 & 0 & -\frac{xw_{\min} + xw_{\max}}{2z_{near}} & 0 \\ 0 & 1 & -\frac{yw_{\min} + yw_{\max}}{2z_{near}} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7-28)$$

뷰잉 좌표의 원점을 투영기준점으로, 뷰 체적의 가까운 평면을 뷰 평면으로 설정하고, 원근투영의 뷰 체적이 대칭형인 경우에는 식 (7-22)를 그대로 사용하면 된다. 만약 같은 조건에서 원근투영의 뷰 체적이 비대칭형인 경우에는 식 (7-22)와 (7-28)을 결합하여 $M_{pp}H$ 를 사용한다.

7.4.6. 원근투영의 정규화 변환

뷰잉 좌표는 원근투영변환행렬을 통해 동차좌표로 변환되고, 각 좌표값을 동차 파라미터 h 로 나누어 실제 투영좌표를 얻을 수 있다. 따라서 이러한 원근투영변환을 통해 절두체의 뷰 체적이 직육면체 뷰 체적으로 변환된다. 이렇게 얻어진 직육면체 뷰 체적을 정규화된 크기의 정육면체로 변환시키는 것이 다음 과정이다.

평행투영에서 사용하였던 것과 같은 정규화 과정을 수행한다. 원근투영 식 (7-22)는 z 축 크기변환 성분을 이미 포함하고 있지만, 대칭형으로 정규화된 정육면체로 변환할 때 그 값을 결정해야 한다. 뷰잉 좌표의 원점을 투영기준점으로 설정하고, 뷰 체적의 가까운 클리핑 평면을 뷰 평면으로 설정($z_{vp} = z_{near}$)한다면, 오른손 좌표계를 사용하는 경우 식 (7-22)에서 $(xw_{min}, yw_{min}, z_{near})$ 에 대한 정규화 변환좌표는 $(-1, -1, 1)$ 이고, $(xw_{max}, yw_{max}, z_{far})$ 에 대한 정규화 변환좌표는 $(1, 1, -1)$ 이 되어야 하므로 다음 관계식을 만족한다.

$$\begin{aligned} z = z_{near} \text{에서 } \frac{z_h}{h} &= \frac{s_z z_{near} + t_z}{-z_{near}} = 1 \\ z = z_{far} \text{에서 } \frac{z_h}{h} &= \frac{s_z z_{far} + t_z}{-z_{far}} = -1 \end{aligned} \quad (7-29)$$

식 (7-29)로부터 z 축에 대한 크기변환과 이동 성분을 구하면 다음과 같다.

$$s_z = -\frac{z_{near} + z_{far}}{z_{near} - z_{far}}, \quad t_z = \frac{2z_{near} z_{far}}{z_{near} - z_{far}} \quad (7-30)$$

또한 x 축과 y 축에 대한 정규화 과정이 필요하다. 직육면체 모양의 뷰 체적의 중심선이 z_{view} 축이 되기 때문에 x 축과 y 축 성분에 대한 이동변환 없이 두 축에 대한 크기변환만 고려하면 된다. 오른손 좌표계를 사용하는 경우 $(xw_{min}, yw_{min}, z_{near})$ 에 대한 정규화 변환좌표는 $(-1, -1, 1)$ 이고, $(xw_{max}, yw_{max}, z_{far})$ 에 대한 정규화 변환좌표는 $(1, 1, -1)$ 이다. 따라서 이 조건을 이용하여 얻어진 변환행렬은 식 (7-31)과 같다.

$$S_{xy} = \begin{bmatrix} \frac{2}{xw_{\max} - xw_{\min}} & 0 & 0 & 0 \\ 0 & \frac{2}{yw_{\max} - yw_{\min}} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7-31)$$

원근투영 뷰 체적이 원래 대칭형의 절두체로 정의되었다면 정규화된 원근변환 행렬 요소는 가시각과 클리핑 윈도의 크기로 표현된다. 따라서 식 (7-27)을 이용하여 관찰 좌표의 원점을 투영기준점으로 설정하고, 뷰 체적의 가까운 평면을 뷰 평면으로 설정한다면 전체적인 변환행렬은 식 (7-32)와 같이 주어진다.

$$M_{sym_pp} = S_{xy} M_{pp}$$

$$= \begin{bmatrix} \frac{-2z_{near}}{xw_{\max} - xw_{\min}} & 0 & 0 & 0 \\ 0 & \frac{-2z_{near}}{yw_{\max} - yw_{\min}} & 0 & 0 \\ 0 & 0 & -\frac{z_{near} + z_{far}}{z_{near} - z_{far}} & \frac{2z_{near}z_{far}}{z_{near} - z_{far}} \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad (7-32)$$

비대칭인 절두체 뷰 체적으로 설정된 경우는 식 (7-28)로 대칭이 되도록 변환한 후 M_{sym_pp} 를 적용하면 된다.

정규화된 좌표계가 왼손 좌표계이며, 가까운 뷰 평면과 먼 뷰 평면의 위치를 왼손 좌표계에서 지정하는 경우에 z_{near} 와 z_{far} 의 부호를 반대로 하고, 투영된 결과를 $S(1, 1, -1)$ 로 크기변환을 하여 변환행렬을 구할 수 있다. 따라서 전체적인 변환행렬 $M_{sym_pp, LH}$ 는 식 (7-33)과 같다.

$$M_{sym_pp, LH} = S(1, 1, -1) M_{sym_pp} H$$

$$= \begin{bmatrix} \frac{2z_{near}}{xw_{\max} - xw_{\min}} & 0 & \frac{xw_{\max} + xw_{\min}}{xw_{\max} - xw_{\min}} & 0 \\ 0 & \frac{2z_{near}}{yw_{\max} - yw_{\min}} & \frac{yw_{\max} + yw_{\min}}{yw_{\max} - yw_{\min}} & 0 \\ 0 & 0 & \frac{z_{near} + z_{far}}{z_{near} - z_{far}} & \frac{2z_{near}z_{far}}{z_{near} - z_{far}} \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad (7-33)$$

이제 나머지 작업은 정규화된 뷰 체적에 대해 클리핑 과정을 적용하는 것이고, 이어서 가시성 검사, 표면 구성 등의 작업이 진행된다.

7.5 3차원 클리핑 알고리즘

앞서 2차원 클리핑 알고리즘을 구현할 때 정규화된 클리핑 윈도 경계를 사용하였다. 마찬가지로 3차원 클리핑 알고리즘은 정규화된 뷰 체적의 경계에 적용한다. 정규화된 뷰 체적의 각 클리핑 경계는 3차원 직교좌표의 평면과 평행이 되며, 클리핑 평면의 좌표범위는 0부터 1 또는 -1부터 1로 설정된다. 따라서 정규화된 뷰 체적은 각 변의 길이가 1 또는 2인 정육면체가 된다. 좌표범위가 -1부터 1인 3차원 클리핑 평면을 위한 평면 방정식은 다음과 같다.

$$xw = \pm 1, \quad yw = \pm 1, \quad zw = \pm 1 \quad (7-34)$$

x 축과 y 축의 클리핑 경계는 정규화된 클리핑 윈도의 경계와 같고, z 축 클리핑 경계는 정규화된 뷰 체적의 가까운 클리핑 평면과 먼 클리핑 평면이 된다.

이와 같이 클리핑 경계가 정의되면 3차원 클리핑 알고리즘을 통해 화면에 표시될 부분을 선택하게 되며, 이에 따라 뷰 체적의 외부에 있는 모든 부분은 제거된다. 일반적으로 3차원 클리핑 알고리즘은 2차원 기법에서 3차원 기법으로 확장한 것이다. 즉, 정규화된 클리핑 윈도의 직선 경계를 사용하는 대신 뷰 체적의 정규화된 경계 평면을 사용한다.

(1) 3차원 영역 코드

가까운 클리핑 평면과 먼 클리핑 평면에 대한 비트를 추가하여 영역 코드를 3차원으로 확장한다. 따라서 [그림 7-22]와 같은 ' $b_5b_4b_3b_2b_1b_0$ '의 6비트 영역 코드를 사용한다. 이 영역 코드의 비트 순서는 왼쪽부터 오른쪽으로 먼(원), 가까운(근), 위(상), 아래(하), 오른쪽(우), 왼쪽(좌) 평면의 영역을 나타낸다. 영역 코드를 구성하는 방법은 6.3절과 같고, 추가로 가까운 평면과 먼 평면에 대한 조건이 추가된다. 투영변환 후 각 점은 동

차좌표 $P = (x_h, y_h, z_h, h)$ 로 표현된다. 정규화된 정육면체의 경계에 대해 클리핑 작업을 수행할 때, 투영좌표가 식 (7-35)의 부등식을 모두 만족하면 해당 좌표는 정규화된 뷰 체적 내부에 존재한다고 판단한다.

b_5	b_4	b_3	b_2	b_1	b_0
(왼)	(근)	(상)	(하)	(우)	(좌)

[그림 7-22] 영역 코드 비트 위치에 해당하는 뷰 체적의 클리핑 경계

$$-1 \leq \frac{x_h}{h} \leq 1, \quad -1 \leq \frac{y_h}{h} \leq 1, \quad -1 \leq \frac{z_h}{h} \leq 1 \quad (7-35)$$

따라서 $h > 0$ 이면 다음과 같은 형식의 부등식을 만들 수 있다.

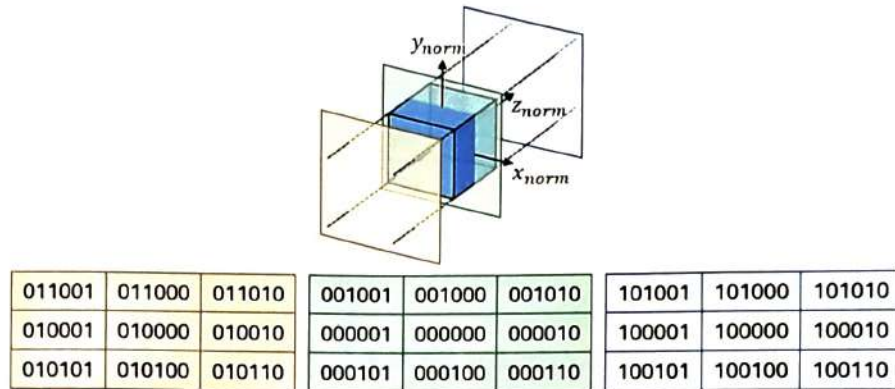
$$-h \leq x_h \leq h, \quad -h \leq y_h \leq h, \quad -h \leq z_h \leq h, \quad \text{if } h > 0 \quad (7-36)$$

이때, 다음과 같은 검사를 통해 해당 영역 코드에 값을 할당한다.

$$\begin{aligned} b_5 &= \begin{cases} 1 & \text{if } h - z_h < 0 \\ 0 & \text{otherwise} \end{cases} \\ b_4 &= \begin{cases} 1 & \text{if } h + z_h < 0 \\ 0 & \text{otherwise} \end{cases} \\ b_3 &= \begin{cases} 1 & \text{if } h - y_h < 0 \\ 0 & \text{otherwise} \end{cases} \\ b_2 &= \begin{cases} 1 & \text{if } h + y_h < 0 \\ 0 & \text{otherwise} \end{cases} \\ b_1 &= \begin{cases} 1 & \text{if } h - x_h < 0 \\ 0 & \text{otherwise} \end{cases} \\ b_0 &= \begin{cases} 1 & \text{if } h + x_h < 0 \\ 0 & \text{otherwise} \end{cases} \end{aligned} \quad (7-37)$$

2차원 클리핑과 같이 간단한 비교 연산을 통해 영역 코드의 비트값을 할당한다. 3차원의 경우 동차좌표가 사용되었기 때문에 단순한 덧셈과 그 결과의 부호를 판단하여 영역 코드의 비트값을 결정한다. 만약 $h < 0$ 인 경우에는 식 (7-36)의 부등호를 반대로 적용하면 된다.

[그림 7-23]은 뷰 체적에 대한 27가지의 영역 코드를 나타낸 것이다.



[그림 7-23] 뷰 체적의 경계에 따른 3차원 6비트 영역 코드

(2) 3차원 점 클리핑

투영기준점 앞의 장면에 정의된 점과 선분은 양수의 동차 파라미터값을 갖는다. 따라서 영역 코드는 식 (7-37)을 이용하여 계산된다. 영역 코드가 설정되면 해당 위치가 뷰 체적의 외부 또는 내부인지 판단할 수 있다. 예를 들어, '000000' 영역 코드는 해당 좌표가 뷰 체적 내부에 있음을 의미한다. 반면에, 그 외의 영역 코드값은 해당 위치가 뷰 체적 외부에 있음을 나타낸다. 즉, 식 (7-37)의 검사에서 한 가지라도 그 값이 음수가 되면 그 점은 뷰 체적 외부에 있다는 것이다.

(3) 3차원 선분 클리핑

간단한 채택 또는 거부를 판단하기 위해 선분 끝점에 대한 영역 코드를 검사한다. 만약 양 끝점의 영역 코드가 모두 '000000'이면 선분의 두 끝점의 영역 코드에 대해 '비트 단위 OR' 논리연산을 수행한 결과가 0이며, 해당 선분 전체가 뷰 체적 내부에 존재한다. 또한 선분의 두 끝점의 영역 코드에 대해 '비트 단위 AND' 논리연산을 수행한 결과가 0이 아닌 경우에는 두 끝점이 특정 클리핑 평면에 대해 밖에 해당되는 영역에 위치하므로 단순히 선분 전체를 제거한다.

만약 앞의 두 가지 외의 경우라면 선분 방정식을 통해 클리핑 경계와 교점을 구하고, 선분의 어느 부분을 제거할지 또는 저장할지 판단해야 한다. 그러므로 교점을 계산

하기 위해 주어진 선분을 방정식으로 표현하고, 제6장의 선분 클리핑 기법을 3차원으로 확장하여 적용한다. 끝점이 $P_1 = (x_1, y_1, z_1)$ 과 $P_2 = (x_2, y_2, z_2)$ 인 선분에 속하는 점 P 는 다음과 같은 매개방정식으로 표현된다.

$$P = P_1 + (P_2 - P_1)u, \quad 0 \leq u \leq 1 \quad (7-38)$$

이 매개방정식에서 $u = 0$ 일 때는 $P = P_1$, $u = 1$ 일 때는 $P = P_2$ 가 된다.

어떤 클리핑 평면과 교점을 형성하는지 판단하기 위해 선분 끝점의 영역 코드를 점 검한다. 두 끝점의 영역 코드를 비교할 때, 어떤 비트의 값이 한 끝점에서는 0이고 다른 끝점에서는 1이라면, 이 선분은 해당 부분의 클리핑 평면과 교점을 이룬다.

여기서 교점을 구하기 위해 식 (7-38)에서 매개변수 u 의 값을 결정한다. 예를 들어, 처리하고 있는 선분과 오른쪽 클리핑 평면의 교점을 구하는 과정을 살펴보면, 이 오른쪽 클리핑 평면은 $x_{\max} = 1$ 로 표현되므로 교점 (x_i, y_i, z_i) 에 해당하는 u 값은 식 (7-38)에 $x_i = 1$ 을 대입하여 다음과 같이 구해진다.

$$x_i = x_1 + (x_2 - x_1)u = 1 \quad (7-39)$$

따라서 교점에 해당하는 u 는 다음과 같다.

$$u = \frac{1 - x_1}{(x_2 - x_1)} \quad (7-40)$$

식 (7-40)을 통해 얻어진 u 값을 그대로 이용하여 이 평면과의 교점에 해당하는 다른 좌표 성분 y_i 와 z_i 값을 구한다. 이때, 좌표 성분이 $|y_i| \leq 1$ 와 $|z_i| \leq 1$ 을 만족하면 이 교점이 뷰 체적 경계 범위 내에 있고, 이 선분은 뷰 체적의 내부를 통과한다고 판단한다. 계속해서 나머지 클리핑 경계에 대해 같은 방법을 적용하여 해당 선분을 확인한다. 모든 경계에 대한 처리가 수행되어야 이 선분에 대한 작업이 완료된다.

선분과 클리핑 평면 및 교점을 구했으나 그 교점이 뷰 체적 내부에 위치하지 않는 경우에는 클리핑 경계 밖의 선분을 제거하고, 나머지 선분에 대하여 다시 선분 클리핑 기법을 적용한다.

3차원 공간에서 선분은 물체를 구성하는 일부분으로 간주된다. 따라서 선분 자체에 대한 독립적인 처리보다는 물체를 구성하는 표면에 대한 클리핑 작업의 일부분으로 선분을 취급하는 경우가 대부분이다.

(4) 3차원 다각형 클리핑

일반적으로 물체는 여러 개의 표면으로 구성되고, 그 표면의 경계는 선형 방정식으로 표현된다. 또한 구나 원기둥같이 곡면으로 구성된 물체도 처리속도를 높이고, 작업의 편리성을 위해 여러 개의 작은 다각형 조각으로 곡면을 근사하여 처리하므로 다각형 클리핑 알고리즘을 많이 사용한다.

물체가 놓인 좌표범위, 위치와 반경 또는 그 밖의 방법으로 좌표 한계를 검사하여 해당 다각형의 전체를 단순히 저장할지 아니면 제거할지 판단한다. 예를 들어, 다각형 전체의 좌표범위가 클리핑 영역 내부로 한정된다면 해당 다각형을 저장한다. 반면에, 전체가 클리핑 영역 외부에 있는 경우에는 그 다각형을 삭제한다.

만약 다각형의 일부는 클리핑 영역 밖에 있고, 일부가 클리핑 영역 내부에 있다면 다각형 전체를 저장하거나 삭제할 수 없다. 이런 경우에는 다각형들에 대해 꼭짓점 목록을 점검해야 한다. 점검과정을 통해 다각형의 일부를 제거하기 위해 일부 모서리를 잘라 내어 꼭짓점 목록 내용을 변경하기도 한다. 또한 어떤 경우에는 새로운 꼭짓점 목록을 추가하여 새로 추가된 표면을 구성하기도 한다.

만약 다각형 표면을 여러 개의 작은 삼각형 조각으로 표현할 수 있다면 다면체와 다각형 표면에 대한 클리핑 작업을 간소화할 수 있다. 3차원 공간에서 삼각형 조각에 대하여 클리핑 작업을 수행할 경우 2차원 평면에서 사용되었던 기법을 적용하여 클리핑 작업을 수행한다. 다만 3차원의 경우 6개의 클리핑 평면에 대해 순서대로 클리핑 작업을 수행한다.

(5) 3차원 곡선 클리핑

곡선으로 구성된 물체 또는 구와 같은 곡면 물체도 그 좌표범위를 점검하여 전체를 단순히 저장할지 아니면 제거할지를 먼저 판단한다. 만약 물체의 일부분은 뷰 체적 내부에 있고 나머지 일부분은 외부에 있다고 판단되는 경우, 곡선형 물체와 클리핑 평면의 교점을 계산하기 위해 곡면 방정식과 클리핑 평면 방정식에 대한 해를 구한다. 일반적인 경우 곡면과 같은 비선형 방정식의 해를 구하는 과정이 복잡하고 계산을 많이 요구한다는 문제점이 있다. 따라서 계산의 복잡도를 줄이기 위해 대부분의 경우 곡면을 여러 개의 다각형 조각으로 근사하여 처리한다. 그러므로 이 경우에도 다각형 클리핑 알고리즘을 적용한다.

7.6 OpenGL의 3차원 뷰잉

3차원 뷰잉을 하려면 먼저 식 (7-4)의 행렬을 이용하여 뷰잉 좌표계로 장면을 변환한다. 이어서 투영을 위한 변환행렬을 목적에 맞게 구성하여 뷰 체적을 정규화한다. 이러한 과정은 변환행렬을 유니폼 변수를 통해 꼭짓점 셰이더에 전달하여 꼭짓점 좌표를 처리하는 것이므로 제5장에서 예시한 기하변환 프로그램과 다를 바 없다.

7.6.1. OpenGL의 뷰잉 변환행렬의 구성

[소스코드 7-1]은 뷰잉 변환을 추가한 기하변환행렬 클래스이다. 3차원 좌표 구조체 `Vec3f`로 표현된 벡터의 내적, 외적, 정규화를 위한 함수 `dotVec3f`, `crossVec3f`, `normalizeVec3f`를 추가하였다. 또한 `TransMat4f`로 표현된 뷰잉 변환을 위한 행렬을 구성하는 함수인 `viewingTrans`를 추가하였다. 이 함수는 71~72행에서 매개변수를 통해 전달받은 뷰 원점(`pOrg`) 및 뷰 기준점(`pRef`)을 이용하여 뷰잉 좌표계의 z 축 방향 단위 벡터인 뷰 평면 법선 벡터 $\mathbf{n}$ 을 구하고, 73~76행에서 이 벡터와 매개변수로 전달된 뷰 업 벡터(`vUp`)를 이용하여 뷰잉 좌표계의 x , y 축 방향 단위 벡터인 $\mathbf{u}$, $\mathbf{v}$ 를 구한 다음, 77~84행에서 식 (7-4)의 행렬을 계산하여 `vMat`에 저장한다.

[소스코드 7-1] (가) `gTransform.h`

```
...    ※ [소스코드 5-1] (가)의 1~17행과 동일함
18    extern TransMat4f* viewingTrans(TransMat4f* vMat, const Vec3f* pOrg,
                                   const Vec3f* pRef, const Vec3f* vUp);
19    extern TransMat4f* orthgonalPrj(TransMat4f* vMat,
                                   float xmin, float xmax,
                                   float ymin, float ymax,
                                   float znear, float zfar);
20    extern TransMat4f* perspectivePrj(TransMat4f* vMat,
                                   float xmin, float xmax,
                                   float ymin, float ymax,
```

```

float znear, float zfar);
21 extern Vec3f* normalizeVec3f(Vec3f* v);
22 extern Vec3f* crossVec3f(Vec3f* v, const Vec3f* a, const Vec3f* b);
23 extern float dotVec3f(const Vec3f* a, const Vec3f* b);

```

[소스코드 7-1] (나) gTransform.c

```

... ※ [소스코드 5-1] (나)의 1~67행과 동일함
68
69 TransMat4f* viewingTrans(TransMat4f* vMat, const Vec3f* pOrg,
                           const Vec3f* pRef, const Vec3f* vUp)
70 {
71     Vec3f n = { pOrg->x - pRef->x, pOrg->y - pRef->y,
                  pOrg->z - pRef->z };
72     normalizeVec3f(&n);
73     Vec3f u, v;
74     crossVec3f(&u, vUp, &n);
75     normalizeVec3f(&u);
76     crossVec3f(&v, &n, &u);
77     vMat->mat[0][0] = u.x; vMat->mat[0][1] = u.y; vMat->mat[0][2] = u.z;
78     vMat->mat[1][0] = v.x; vMat->mat[1][1] = v.y; vMat->mat[1][2] = v.z;
79     vMat->mat[2][0] = n.x; vMat->mat[2][1] = n.y; vMat->mat[2][2] = n.z;
80     vMat->mat[0][3] = -dotVec3f(&u, pOrg);
81     vMat->mat[1][3] = -dotVec3f(&v, pOrg);
82     vMat->mat[2][3] = -dotVec3f(&n, pOrg);
83     vMat->mat[3][0] = vMat->mat[3][1] = vMat->mat[3][2] = 0.0f;
84     vMat->mat[3][3] = 1.0f;
85     return vMat;
86 }
87
88 TransMat4f* orthogonalPrj(TransMat4f* vMat, float xmin, float xmax,
                            float ymin, float ymax,
                            float znear, float zfar) {
89     vMat->mat[0][0] = 2.0f / (xmax - xmin);

```

```

90     vMat->mat[0][1] = vMat->mat[0][2] = 0.0f;
91     vMat->mat[0][3] = -(xmax + xmin) / (xmax - xmin);
92     vMat->mat[1][0] = vMat->mat[1][2] = 0;
93     vMat->mat[1][1] = 2.0f / (ymax - ymin);
94     vMat->mat[1][3] = -(ymax + ymin) / (ymax - ymin);
95     vMat->mat[2][0] = vMat->mat[2][1] = 0.0f;
96     vMat->mat[2][2] = 2.0f / (znear - zfar);
97     vMat->mat[2][3] = (znear + zfar) / (znear - zfar);
98     vMat->mat[3][0] = vMat->mat[3][1] = vMat->mat[3][2] = 0.0f;
99     vMat->mat[3][3] = 1.0f;
100    return vMat;
101 }
102
103 TransMat4f* perspectivePrj(TransMat4f* vMat, float xmin, float xmax,
                             float ymin, float ymax,
                             float znear, float zfar)
104 {
105     identity_GTM(vMat);
106
107     vMat->mat[0][0] = 2.0 * znear / (xmax - xmin);
108     vMat->mat[0][2] = (xmax + xmin) / (xmax - xmin);
109     vMat->mat[1][1] = 2.0 * znear / (ymax - ymin);
110     vMat->mat[1][2] = (ymax + ymin) / (ymax - ymin);
111     vMat->mat[2][2] = (znear + zfar) / (znear - zfar);
112     vMat->mat[2][3] = 2.0f * znear * zfar / (znear - zfar);
113     vMat->mat[3][2] = -1.0;
114     vMat->mat[3][3] = 0;
115     return vMat;
116 }
117
118 Vec3f* normalizeVec3f(Vec3f* v)
119 {
120     float len = sqrt(v->x * v->x + v->y * v->y + v->z * v->z);
121     v->x /= len;

```



```

122     v->y /= len;
123     v->z /= len;
124     return v;
125 }
126
127 Vec3f* crossVec3f(Vec3f* v, const Vec3f* a, const Vec3f* b)
128 {
129     v->x = a->y * b->z - a->z * b->y;
130     v->y = a->z * b->x - a->x * b->z;
131     v->z = a->x * b->y - a->y * b->x;
132     return v;
133 }
134
135 float dotVec3f(const Vec3f* a, const Vec3f* b)
136 {
137     return a->x * b->x + a->y * b->y + a->z * b->z;
138 }

```

7.6.2. OpenGL의 직교투영

[소스코드 7-1] (나)의 88~101행에 정의된 함수 `orthogonalPrj`는 매개변수로 전달된 클리핑 원도의 x , y , z 축 경계 좌표를 이용하여 직교투영 정규화 변환행렬을 구성한다. 단, OpenGL에서는 최종 단계의 정규좌표계가 왼손 좌표계를 사용하므로 식 (7-7)의 행렬을 사용한다. 전체적인 직교투영행렬은 장면의 각 좌표에 뷰잉 변환을 한 후 직교투영을 하는 것이므로, 이 변환행렬을 순서대로 곱하여 복합변환행렬을 만들면 된다.

[소스코드 7-2]의 렌더링 콜백 함수 `RenderCB`에서는 111~115행에서 `viewMat`에 뷰잉 변환행렬을 구성하였다. 뷰 원점 (120, 80, 100)에서 뷰 기준점 (0, 0, 0)을 향해 바라보도록 하였고, 뷰업 벡터는 y 축 방향인 (0, 1, 0)을 지정하였다.

117행에서는 `orthoMat`에 직교투영을 위한 변환행렬을 구하여 저장한다. 가까운 클리핑 평면을 뷰 평면으로 사용하였으며, 클리핑 경계는 x 축과 y 축에 대해 -100과 100, 왼손 좌표계의 z 축에 대해 50과 300으로 지정하였다.

118~119행에서는 `orthoMat`과 `viewMat`을 곱하여 전체적인 변환행렬을 구한다. 이때, 변환 순서에 맞게 행렬을 배치하여 곱해야 한다. 120행에서는 이 변환행렬을 꼭짓점 셰이더의 유니폼 변수에 전달한다.

이제 뷰잉을 위한 준비가 끝났으며, 139~175행의 `InitVBOs` 함수에서 준비해 놓은 육면체의 꼭짓점 데이터를 이용하여 렌더링을 지시한다. [그림 7-24]는 이 프로그램의 실행결과가 출력된 화면이다.

[소스코드 7-2] OrthogonalPrj.c

```

1  #include <stdio.h>
2  #include <math.h>
3
4  #include <gl/glew.h>
5  #include <gl/freeglut.h>
6  #include "gTransform.h"
7
8  typedef struct {    // 색을 표현하기 위한 구조체
9      float r, g, b, a;
10 } VecColor4f;
11
12 typedef struct {
13     Vec3f      mCoord;
14     VecColor4f mColor;
15 } Vertex;
16
17 enum { CUBE, N_VBOs };
18 GLuint VBO[N_VBOs];    // 꼭짓점 버퍼 객체
19 GLuint uGTransLocation; // 기하변환 행렬 유니폼의 위치
20
21 static const char* pVS =    // 꼭짓점 셰이더 소스
...     ※ [소스코드 5-1] (다)의 꼭짓점 셰이더와 동일함
33
34 static const char* pFS =    // 프래그먼트 셰이더 소스
...     ※ [소스코드 5-1] (다)의 프래그먼트 셰이더와 동일함

```

```

43
44 void AddShader(GLuint shaderProg, const char* pShaderSrc,
               GLint ShaderType)
45 { // 셰이더 생성
...   ※ [소스코드 5-1] (다)의 AddShader 함수와 동일함
66 }
67
68 void SetUpShaders( )
69 { // 셰이더 프로그램 객체 생성
...   ※ [소스코드 5-1] (다)의 SetUpShaders 함수와 동일함
104 }
105
106 void RenderCB( )
107 {
108     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
109     glClear(GL_COLOR_BUFFER_BIT); // 백색으로 화면 지움
110
111     TransMat4f viewMat;
112     Vec3f pOrg = { 120.0f, 80.0f, 100.0f };
113     Vec3f pRef = { 0.0f, 0.0f, 0.0f };
114     Vec3f vUp = { 0.0f, 1.0f, 0.0f };
115     viewingTrans(&viewMat, &pOrg, &pRef, &vUp);
116     TransMat4f orthoMat;
117     orthgonalPrj(&orthoMat, -100.0f, 100.0f,
                  -100.0f, 100.0f, 50.0f, 300.0f);
118     TransMat4f tMat = orthoMat;
119     matmul_GTM(&tMat, &viewMat);
120     glUniformMatrix4fv(uGTransLocation, 1, GL_TRUE, tMat.mat);
121
122     glEnableVertexAttribArray(0);
123     glEnableVertexAttribArray(1);
124     glBindBuffer(GL_ARRAY_BUFFER, VBO[CUBE]);
125     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), 0);
126     glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, sizeof(Vertex),

```

```

127         (const GLvoid*)sizeof(Vec3f));
128     glDrawArrays(GL_LINE_LOOP, 0, 4);
129     glDrawArrays(GL_LINE_LOOP, 4, 4);
130     glDrawArrays(GL_LINE_LOOP, 8, 4);
131     glDrawArrays(GL_LINE_LOOP, 12, 4);
132     glDrawArrays(GL_LINE_LOOP, 16, 4);
133     glDrawArrays(GL_LINE_LOOP, 20, 4);
134     glDisableVertexAttribArray(0);
135     glDisableVertexAttribArray(1);
136
137     glFinish();
138 }
139
140 void InitVB0s()
141 {
142     Vertex Vertices[24] = { // 육면체의 꼭짓점 좌표 및 색
143         { {-50.0f, -50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
144         { {-50.0f, 50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
145         { {50.0f, 50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
146         { {50.0f, -50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
147
148         { {-50.0f, -50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
149         { {50.0f, -50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
150         { {50.0f, 50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
151         { {-50.0f, 50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
152
153         { {-50.0f, -50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
154         { {-50.0f, -50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
155         { {-50.0f, 50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
156         { {-50.0f, 50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
157
158         { {50.0f, -50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
159         { {50.0f, 50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
160         { {50.0f, 50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
161         { {50.0f, -50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} }
162     };
163 }

```



```

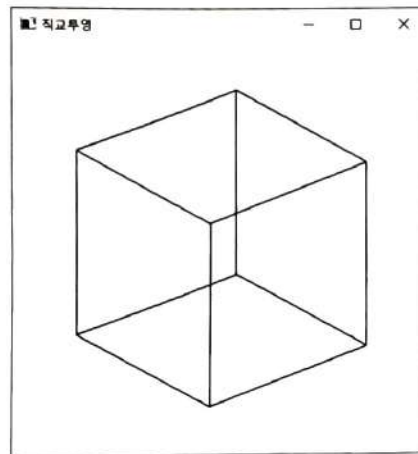
160         { {50.0f, -50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
161
162         { {-50.0f, -50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
163         { {50.0f, -50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
164         { {50.0f, -50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
165         { {-50.0f, -50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
166
167         { {-50.0f, 50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
168         { {-50.0f, 50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
169         { {50.0f, 50.0f, 50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} },
170         { {50.0f, 50.0f, -50.0f}, {0.0f, 0.0f, 0.0f, 1.0f} } };
171
172     glGenBuffers(N_VBOs, VBO);
173     glBindBuffer(GL_ARRAY_BUFFER, VBO[CUBE]);
174     glBufferData(GL_ARRAY_BUFFER, sizeof(Vertices), Vertices,
175                 GL_STATIC_DRAW);
176
177     int main(int argc, char** argv)
178     {
179         glutInit(&argc, argv);
180         glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
181         glutInitWindowPosition(50, 100);
182         glutInitWindowSize(400, 400);
183         glutCreateWindow("원근투영");
184
185         GLError s = glewInit();
186         if (s != GLEW_OK) {
187             fprintf(stderr, "오류 - %s\n", glewGetErrorString(s));
188             return 1;
189         }
190
191         SetUpShaders();
192         InitVBOs();

```

```

193
194     glutDisplayFunc(RenderCB);
195     glutMainLoop();
196
197     return 0;
198 }

```



[그림 7-24] 정육면체를 직교투영하는 OpenGL 프로그램의 실행결과

7.6.3. OpenGL의 원근투영

[소스코드 7-1] (나)의 103~116행에 정의된 함수 `perspectivePrj`는 매개변수로 전달된 가까운 뷰 평면의 x , y 축 클리핑 경계 좌표와 z 축의 뷰 범위를 이용하여 식 (7-33)의 원근투영 정규화 변환행렬을 구성한다. 단, 왼손 좌표계를 사용하는 정규좌표를 고려한다. 전체적인 원근투영행렬은 장면의 각 좌표에 뷰잉 변환을 한 후 원근투영을 하는 것이므로, 이 변환행렬들을 순서대로 곱하여 복합변환행렬을 만들면 된다.

[소스코드 7-3]의 렌더링 콜백 함수 `RenderCB`에서는 111~115행에서 `viewMat`에 뷰잉 변환행렬을 저장하도록 하였다. 뷰 원점 (150, 120, 100)에서 뷰 기준점 (0, 0, 0)을 향해 바라보도록 하였고, 뷰업 벡터는 y 축 방향인 (0, 1, 0)을 지정하였다.

117행에서는 `perspectiveMat`에 원근투영을 위한 변환행렬을 구하여 저장한다. 가까

운 클리핑 평면을 뷰 평면으로 사용하였으며, 뷰 평면의 클리핑 경계는 x 축과 y 축에 대해 -50 과 50 , 클리핑 평면의 왼손 좌표계 z 축 좌표는 70 과 350 으로 지정하였다.

118~119행에서는 `perspectiveMat`과 `viewMat`의 행렬 곱을 함으로써 전체적인 변환 행렬 `tMat`을 구한다. 120행에서는 이 변환행렬을 꼭짓점 셰이더의 유니폼 변수에 전달한다.

이제 뷰잉을 위한 준비가 끝났으며, [소스코드 7-2]와 동일한 절차에 따라 렌더링을 실행한다. [그림 7-25]는 이 프로그램의 실행결과가 출력된 화면이다.

[소스코드 7-3] PerspectivePrj.c

```

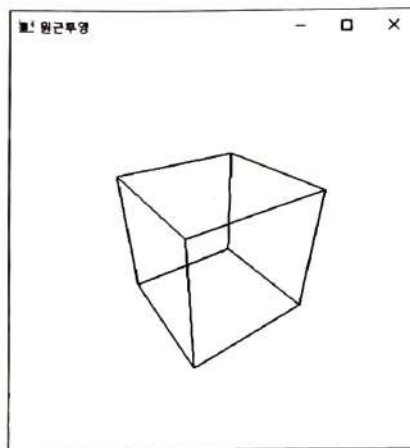
...   ※ 1~105행까지 [소스코드 7-2]와 동일함
106   void RenderCB( )
107   {
108       glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
109       glClear(GL_COLOR_BUFFER_BIT);    // 백색으로 화면 지움
110
111       TransMat4f viewMat;
112       Vec3f pOrg = { 150.0f, 120.0f, 100.0f };
113       Vec3f pRef = { 0.0f, 0.0f, 0.0f };
114       Vec3f vUp = { 0.0f, 1.0f, 0.0f };
115       viewingTrans(&viewMat, &pOrg, &pRef, &vUp);
116       TransMat4f perspectiveMat;
117       perspectivePrj(&perspectiveMat, -50.0f, 50.0f,
                     -50.0f, 50.0f, 70.0f, 350.0f);
118       TransMat4f tMat = perspectiveMat;
119       matmul_GTM(&tMat, &viewMat);
120       glUniformMatrix4fv(uGTransLocation, 1, GL_TRUE, tMat.mat);
121
122       glEnableVertexAttribArray(0);
123       glEnableVertexAttribArray(1);
124       glBindBuffer(GL_ARRAY_BUFFER, VBO[CUBE]);
125       glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), 0);
126       glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, sizeof(Vertex),
                             (const GLvoid*)sizeof(Vec3f));

```

```

127     glDrawArrays(GL_LINE_LOOP, 0, 4);
128     glDrawArrays(GL_LINE_LOOP, 4, 4);
129     glDrawArrays(GL_LINE_LOOP, 8, 4);
130     glDrawArrays(GL_LINE_LOOP, 12, 4);
131     glDrawArrays(GL_LINE_LOOP, 16, 4);
132     glDrawArrays(GL_LINE_LOOP, 20, 4);
133     glDisableVertexAttribArray(0);
134     glDisableVertexAttribArray(1);
135
136     glFinish();
137 }
... ※ 이하 [소스코드 7-2]와 유사함

```



[그림 7-25] 정육면체를 원근투영하는 OpenGL 프로그램의 실행결과

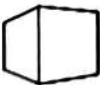
요약

- 3차원 뷰잉 파이프라인: 모델링 변환 → 뷰잉 변환 → 투영변환 → 정규화 및 클리핑 → 뷰포트 변환
- 뷰잉 좌표계는 관찰자의 눈 위치에 해당되는 뷰 원점, 뷰 평면의 방향을 나타내는 뷰 평면 법선 벡터, 뷰 평면의 위쪽 방향을 나타내는 뷰업 벡터를 통해 설정할 수 있다.
- 뷰잉 변환은 뷰 원점을 세계좌표 원점으로 이동하는 변환과 이동된 뷰잉 좌표계의 세 좌표축을 세계좌표계 좌표축에 일치하도록 회전하는 변환으로 구성된다.
- 평행투영은 좌표점을 평행한 투영선을 따라 뷰 평면에 투영하는 방식으로, 물체의 비례 관계가 그대로 유지되며 평행한 선은 투영 후에도 평행하다.
- 평행투영에는 투영선이 뷰 평면에 수직인 직교투영과 수직이 아닌 경사평행투영이 있다.
- 직교투영의 뷰 체적은 직육면체 형태이며, 정면도, 측면도, 평면도, 측각직교투영 등이 있다.
- 경사평행투영의 뷰 체적은 평행육면체 형태이며, 캐버리어 투영이나 캐비닛 투영이 경사평행투영에 속한다.
- 원근투영은 투영기준점으로 수렴하는 경로를 따라 뷰 평면에 물체를 투영하는 방법으로, 물체는 원근법에 의해 표시된다.
- 서로 평행한 선분들이 뷰 평면과 평행하지 않다면 원근투영 결과 이 평행 선분들의 연장선은 소실점으로 모인다.
- 원근투영의 뷰 체적은 사각뿔 절두체 형태이다.
- 원근투영변환은 아핀 변환이 아니며, 변환된 동차좌표의 동차 파라미터에 z 축 좌표에 해당되는 값이 포함된다.
- OpenGL에서 뷰잉 및 투영 변환을 하려면 적절한 변환행렬을 구성하여 유니폼 변수를 통해 꼭짓점 셰이더에 전달한다. 꼭짓점 셰이더는 이 변환행렬을 이용하여 꼭짓점 좌표를 변환한다.



연습문제

객관식

- 다음 중 뷰잉 좌표계의 y_{view} 축 방향과 관련이 있는 것은?
 - ① 뷰 지점
 - ② 뷰 평면 법선 벡터
 - ③ 뷰업 벡터
 - ④ 뷰 기준점
- 다음 중 경사평행투영에 해당되는 것은?
 - ① 등각투영
 - ② 정면도
 - ③ 평면도
 - ④ 캐버리어 투영
- 직교투영의 뷰 체적 형태는?
 - ① 삼각뿔
 - ② 직육면체
 - ③ 사각뿔 절두체
 - ④ 원뿔
- 원근투영에서 투영기준점이 뷰 평면에 가까워지면 어떠한 투영효과를 얻을 수 있는가?
 - ① 원근효과가 강조된다.
 - ② 원근효과가 감소한다.
 - ③ 축 소실점 수가 증가한다.
 - ④ 계산시간이 감소한다.
- 정육면체에 대한 투영도 중 축 소실점이 2개인 2점 투영은?
 - ① 
 - ② 
 - ③ 
 - ④ 

6. 다음 중 평행투영에 대한 설명을 모두 모은 것은?

- (가) 평행선을 따라 물체를 투영한다.
(나) 물체의 평행한 선은 투영 결과에서도 평행하다.
(다) 원근감을 표현할 수 있다.
(라) 투영 경로가 한 점에 모인다.

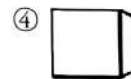
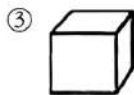
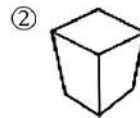
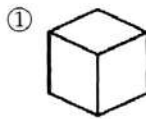
- ① (¬), (↓) ② (¬), (↔)
- ③ (↓), (⇒) ④ (↔), (⇒)

7. 3차원 뷰잉 좌표계를 설정하기 위한 파라미터와 용도가 올바르게 연결된 것은?

- | | |
|----------------|---------------------|
| (ㄱ) 뷰 지점 | (1) 눈의 위치 지정 |
| (ㄴ) 뷰 평면 법선 벡터 | (2) 뷰 평면의 방향 지정 |
| (ㄷ) 뷰업 벡터 | (3) 뷰 평면의 위쪽 방향을 지정 |

- ① $(\neg)-(2), (\neg)-(1), (\neg)-(3)$ ② $(\neg)-(3), (\neg)-(2), (\neg)-(1)$
③ $(\neg)-(1), (\neg)-(2), (\neg)-(3)$ ④ $(\neg)-(1), (\neg)-(3), (\neg)-(2)$

8. 다음 중 정육면체에 대한 축각직교투영에 해당되는 것은?



9. 소실점이 2개인 투영에 대한 설명은?

- ① 경사평행투영의 결과이다.
- ② 부 체적의 형태는 직육면체이다.
- ③ 부 평면이 2개의 주축과 교차한다.
- ④ 원근감을 표현할 수 없다.

정답 1. ③ 2. ④ 3. ② 4. ① 5. ③ 6. ① 7. ③ 8. ① 9. ③

주관식

1. 3차원 뷰잉 파이프라인에서 각각의 변환이 어떠한 용도로 사용되는지 설명하라.
2. 뷰 지점, 뷰 평면 법선 벡터, 뷰업 벡터가 표현하는 뷰잉 좌표계에 대해 설명하라.
3. 3차원 정규화된 뷰 체적에 대해 3차원 직선이 클리핑 처리될 수 있도록 Cohen-Sutherland의 2차원 선 클리핑 알고리즘을 확장하라.
4. 3차원 정규화된 뷰 체적에 대해 3차원 직선이 클리핑 처리될 수 있도록 Liang-Barsky의 2차원 선 클리핑 알고리즘을 확장하라.
5. [소스코드 7-3]을 수정하여 1점 투영 및 2점 투영을 하는 프로그램을 작성하라.